

CS 4782: Midterm Review

Akanksha, Andrew, Daniel, Jane, Jiasheng, William

Questions

If you have a quick clarifying question or something to point out, you can ask, but **please save longer questions for our Q&A after** so that we have time to get through all the material

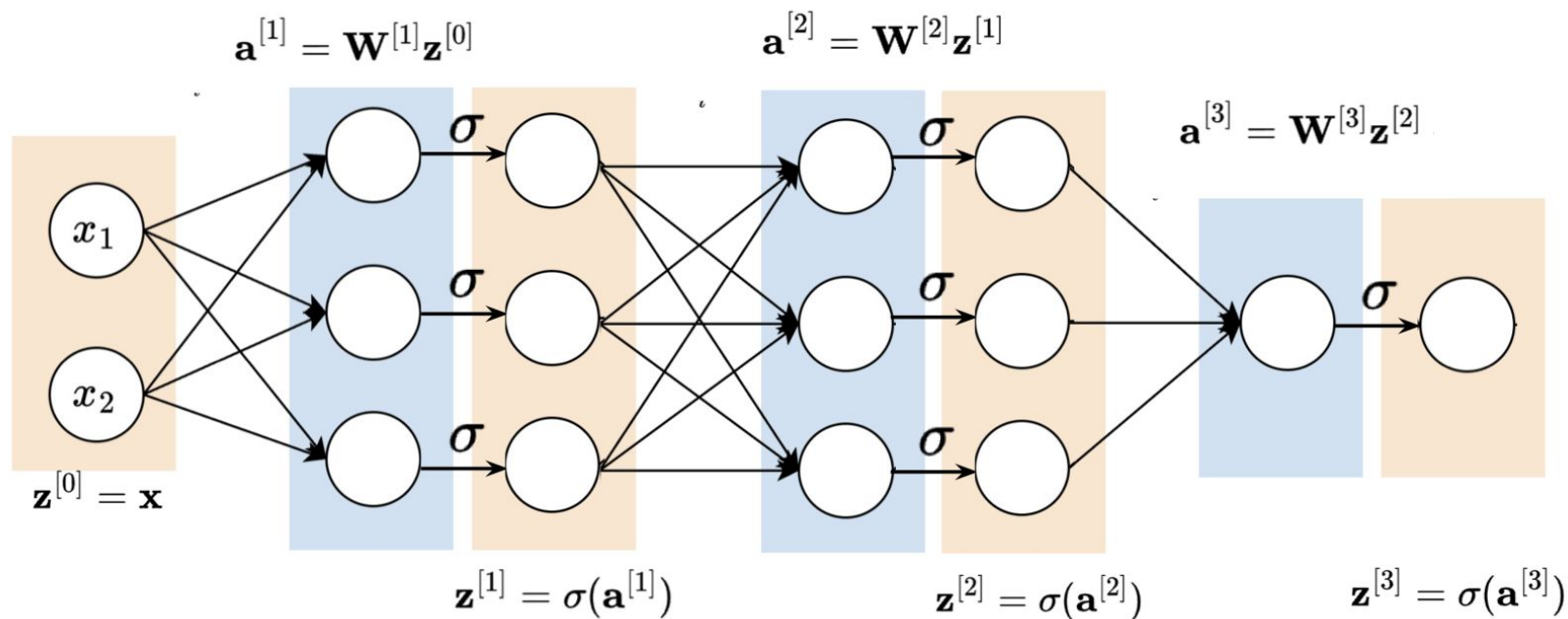
Agenda

1. MLPs & Backprop
2. Optimization
3. Regularization
4. Word Embeddings
5. RNNs & LSTMs
6. Transformers & Attention
7. CNNs and U-Nets
8. Contrastive Learning
9. Vision Transformers
10. GANs
11. VAEs
12. Diffusion Models

1. MLPs & Backprop

Deep learning model after....	
Epoch 1	
Epoch 10	
Epoch 99	

MLPs: Forward Pass



MLPs: Backpropagation

Algorithm Backward Pass through MLP (Detailed)

- 1: **Input:** $\{\mathbf{z}^{[1]}, \dots, \mathbf{z}^{[L]}\}, \{\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}\}$, loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}}$
- 2: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \frac{\partial \mathbf{z}^{[L]}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \odot \sigma^{[L]'}(\mathbf{a}^{[L]})$ ▷ Error term
- 3: **for** $l = L$ **to** 1 **do**
- 4: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{z}^{[l-1]})^T$ ▷ Gradient of weights
- 5: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$ ▷ Gradient of biases
- 6: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l-1]}} = (\mathbf{W}^{[l]})^T \delta^{[l]}$
- 7: $\delta^{[l-1]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} \frac{\partial \mathbf{z}^{[l-1]}}{\partial \mathbf{a}^{[l-1]}} = ((\mathbf{W}^{[l]})^T \delta^{[l]}) \odot \sigma^{[l-1]'}(\mathbf{a}^{[l-1]})$
- 8: **end for**
- 9: **Output:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1:L]}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1:L]}}$

MLPs: Backpropagation

Algorithm Backward Pass through MLP (Detailed)

- 1: **Input:** $\{\mathbf{z}^{[1]}, \dots, \mathbf{z}^{[L]}\}, \{\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}\},$ loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}}$
 - 2: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \frac{\partial \mathbf{z}^{[L]}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \odot \sigma^{[L]'}(\mathbf{a}^{[L]})$ ▷ Error term
 - 3: **for** $l = L$ **to** 1 **do**
 - 4: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{z}^{[l-1]})^T$ ▷ Gradient of weights
 - 5: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$ ▷ Gradient of biases
 - 6: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l-1]}} = (\mathbf{W}^{[l]})^T \delta^{[l]}$
 - 7: $\delta^{[l-1]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} \frac{\partial \mathbf{z}^{[l-1]}}{\partial \mathbf{a}^{[l-1]}} = ((\mathbf{W}^{[l]})^T \delta^{[l]}) \odot \sigma^{[l-1]'}(\mathbf{a}^{[l-1]})$
 - 8: **end for**
 - 9: **Output:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1:L]}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1:L]}}$
-

MLPs: Backpropagation

Algorithm Backward Pass through MLP (Detailed)

- 1: **Input:** $\{\mathbf{z}^{[1]}, \dots, \mathbf{z}^{[L]}\}, \{\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}\}$, loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}}$
- 2: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \frac{\partial \mathbf{z}^{[L]}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \odot \sigma^{[L]'}(\mathbf{a}^{[L]})$ ▷ Error term
- 3: **for** $l = L$ **to** 1 **do**
- 4: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{z}^{[l-1]})^T$ ▷ Gradient of weights
- 5: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$ ▷ Gradient of biases
- 6: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l-1]}} = (\mathbf{W}^{[l]})^T \delta^{[l]}$
- 7: $\delta^{[l-1]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} \frac{\partial \mathbf{z}^{[l-1]}}{\partial \mathbf{a}^{[l-1]}} = ((\mathbf{W}^{[l]})^T \delta^{[l]}) \odot \sigma^{[l-1]'}(\mathbf{a}^{[l-1]})$
- 8: **end for**
- 9: **Output:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1:L]}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1:L]}}$

MLPs: Backpropagation

Algorithm Backward Pass through MLP (Detailed)

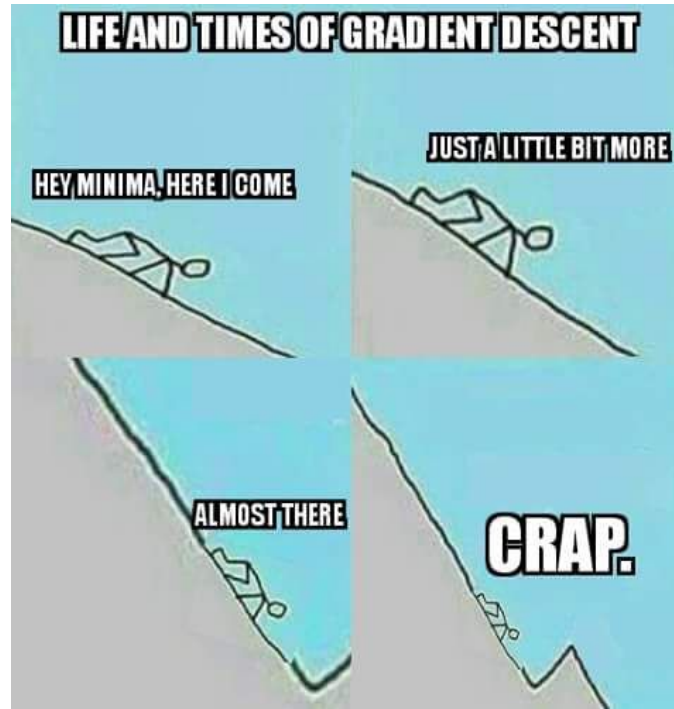
- 1: **Input:** $\{\mathbf{z}^{[1]}, \dots, \mathbf{z}^{[L]}\}, \{\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}\}$, loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}}$
 - 2: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \frac{\partial \mathbf{z}^{[L]}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \odot \sigma^{[L]'}(\mathbf{a}^{[L]})$ ▷ Error term
 - 3: **for** $l = L$ **to** 1 **do**
 - 4: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{z}^{[l-1]})^T$ ▷ Gradient of weights
 - 5: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$ ▷ Gradient of biases
 - 6: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l-1]}} = (\mathbf{W}^{[l]})^T \delta^{[l]}$
 - 7: $\delta^{[l-1]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} \frac{\partial \mathbf{z}^{[l-1]}}{\partial \mathbf{a}^{[l-1]}} = ((\mathbf{W}^{[l]})^T \delta^{[l]}) \odot \sigma^{[l-1]'}(\mathbf{a}^{[l-1]})$
 - 8: **end for**
 - 9: **Output:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1:L]}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1:L]}}$
-

MLPs: Backpropagation

Algorithm Backward Pass through MLP (Detailed)

- 1: **Input:** $\{\mathbf{z}^{[1]}, \dots, \mathbf{z}^{[L]}\}, \{\mathbf{a}^{[1]}, \dots, \mathbf{a}^{[L]}\}$, loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}}$
 - 2: $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \frac{\partial \mathbf{z}^{[L]}}{\partial \mathbf{a}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \odot \sigma^{[L]'}(\mathbf{a}^{[L]})$ ▷ Error term
 - 3: **for** $l = L$ **to** 1 **do**
 - 4: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{z}^{[l-1]})^T$ ▷ Gradient of weights
 - 5: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$ ▷ Gradient of biases
 - 6: $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l]}} \frac{\partial \mathbf{a}^{[l]}}{\partial \mathbf{z}^{[l-1]}} = (\mathbf{W}^{[l]})^T \delta^{[l]}$
 - 7: $\delta^{[l-1]} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[l-1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l-1]}} \frac{\partial \mathbf{z}^{[l-1]}}{\partial \mathbf{a}^{[l-1]}} = ((\mathbf{W}^{[l]})^T \delta^{[l]}) \odot \sigma^{[l-1]'}(\mathbf{a}^{[l-1]})$
 - 8: **end for**
 - 9: **Output:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1:L]}}, \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1:L]}}$
-

2. Optimization



Optimization

Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{1}{n} \sum_{i=1}^n \nabla \ell(\mathbf{w}_t, \mathbf{x}_i)$$

steady convergence
but $O(nT)$ too slow

Stochastic Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla \ell(\mathbf{w}_t, \mathbf{x}_i)$$

$O(T)$ but trade-off convergence
noise ball

Minibatch SGD

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla \ell(\mathbf{w}_t, \mathbf{x}_i)$$

reduce the size of the noise ball
 $O(bT)$

SGD w. Momentum

$$m_{t+1} = \mu m_t - \alpha \nabla l(w_t; x_i)$$

$$w_{t+1} = w_t + m_{t+1}$$

accumulate gradients
accelerate convergence (reduce T)
smoother path

Optimization

adjust learning rate to handle sparse gradient
but easily captured by saddle points

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{g}_t^2$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1} + \epsilon}} \odot \mathbf{g}_t$$

Adagrad

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \mathbf{g}_t^2$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{\mathbf{v}_{t+1} + \epsilon}} \odot \mathbf{g}_t$$

RMSProp

slow the decay of the learning rate
harder to be captured by saddle points

$$\mathbb{E}[\hat{\mathbf{m}}_{t+1}]$$

$$\mathbb{E}[\hat{\mathbf{v}}_{t+1}]$$

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}$$

$$\hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}} \odot \hat{\mathbf{m}}_{t+1}$$

ADAM

(Adaptive Moment Estimate)

momentum + RMSProp

expectation terms prevents from shrinking by parameters
we will have better Adam (AdamW) from weight decay!

Optimization

Hyperparameters:

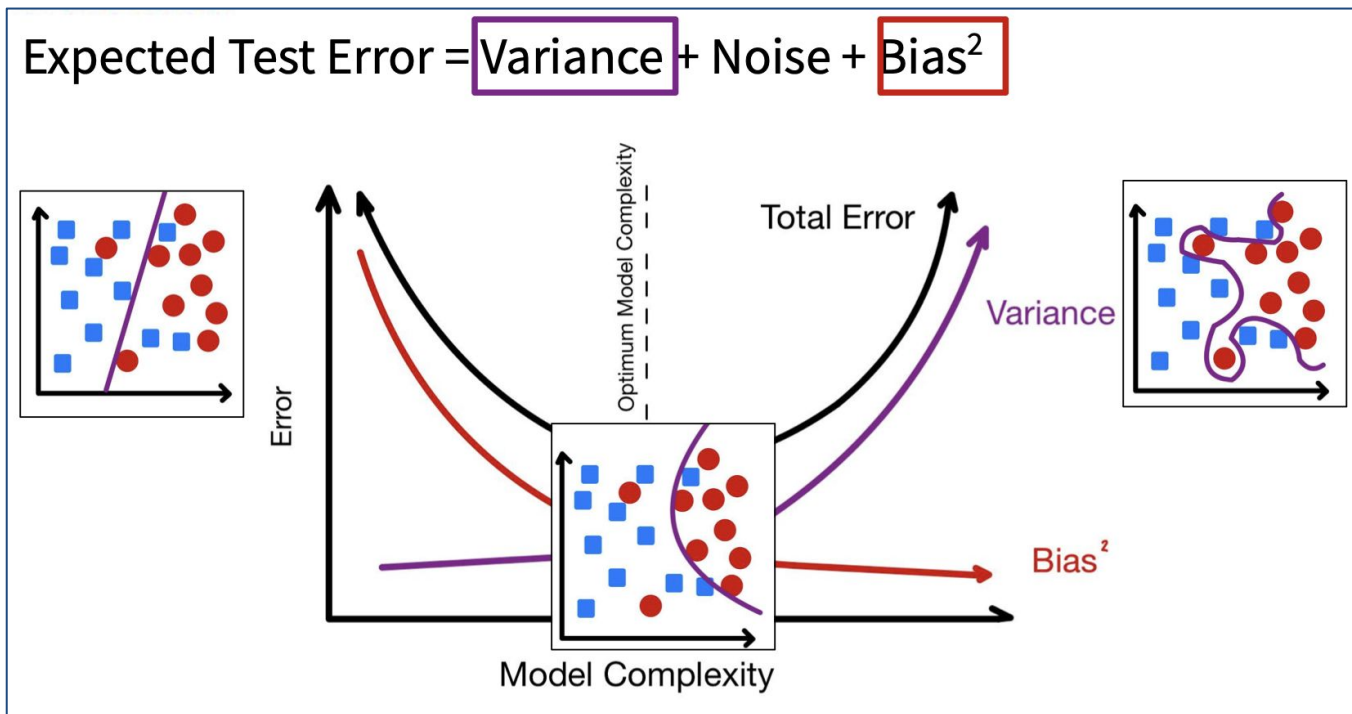
- Learning rate
- Batch Size
- Betas for momentum and RMSProp
- Regularization Parameter

3. Regularization

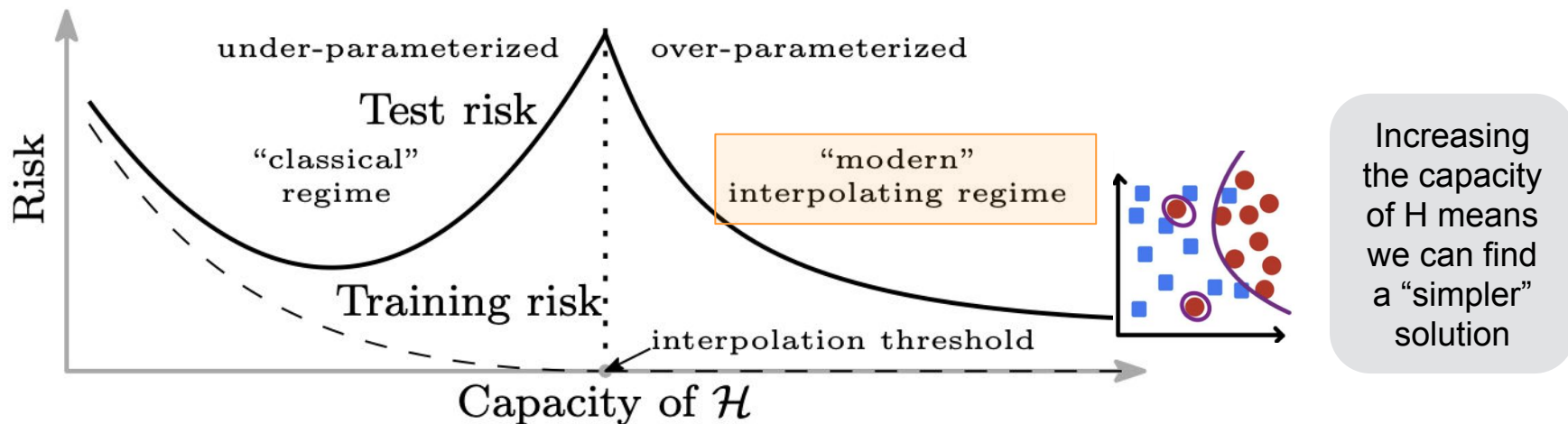


What is regularization?

Broadly: ways to prevent models from overfitting

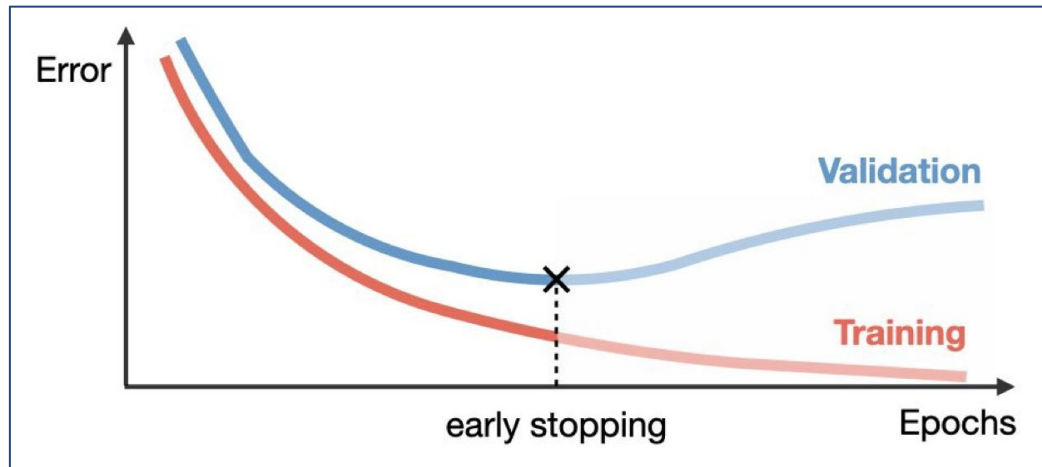


In practice: neural networks exhibit Deep Double Descent



Regularization

- Early Stopping
- L1 and L2 Regularization
- Data Augmentation
- DropOut



Stop training at point with best validation loss (to generalize better)

Regularization

- Early Stopping
- L1 and L2 Regularization
- Data Augmentation
- DropOut

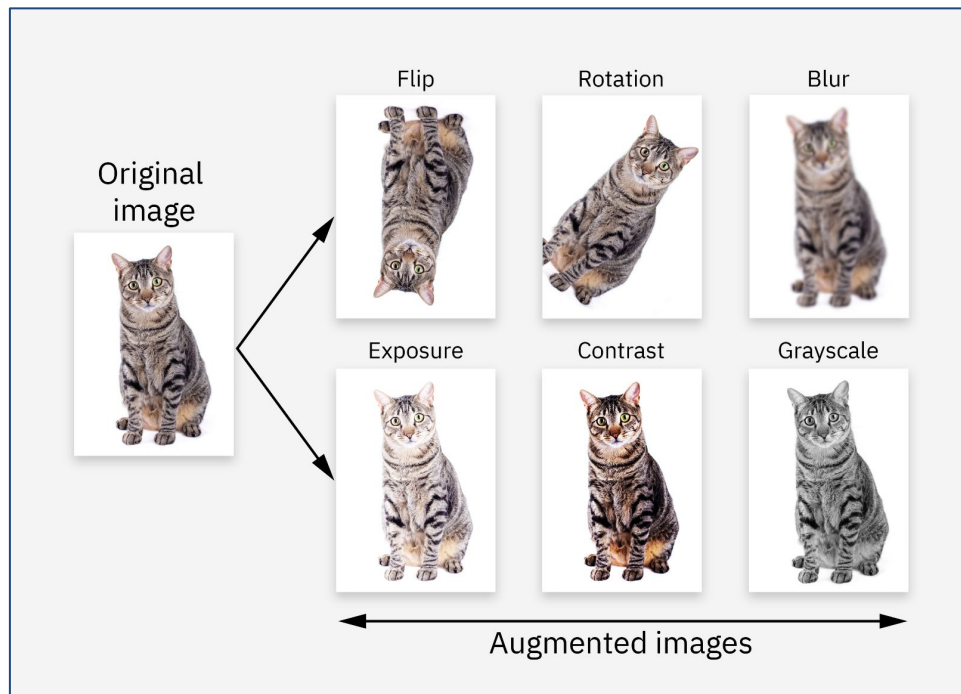
Optimization Problem

L1:
$$\operatorname{argmin}_{\mathbf{w}} \mathcal{L}(\mathbf{w}, D) + \lambda |\mathbf{w}|$$

L2:
$$\operatorname{argmin}_{\mathbf{w}} \mathcal{L}(\mathbf{w}, D) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

Regularization

- Early Stopping
- L1 and L2 Regularization
- Data Augmentation
- DropOut



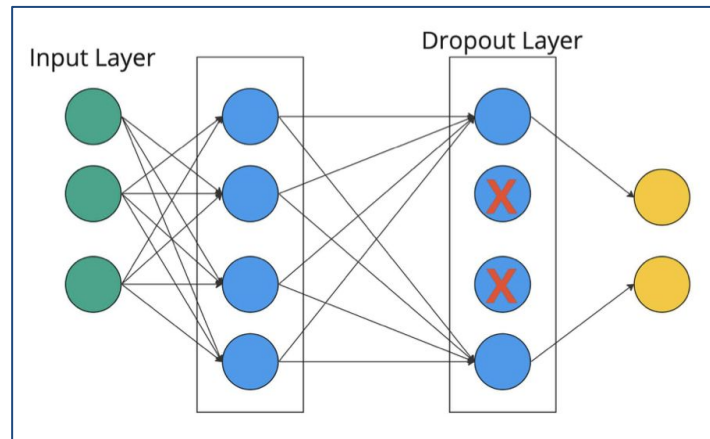
Make model more robust
without annotating more data

Regularization

- Early Stopping
- L1 and L2 Regularization
- Data Augmentation
- DropOut

p = probability of
keeping a neuron during
training, commonly 0.5

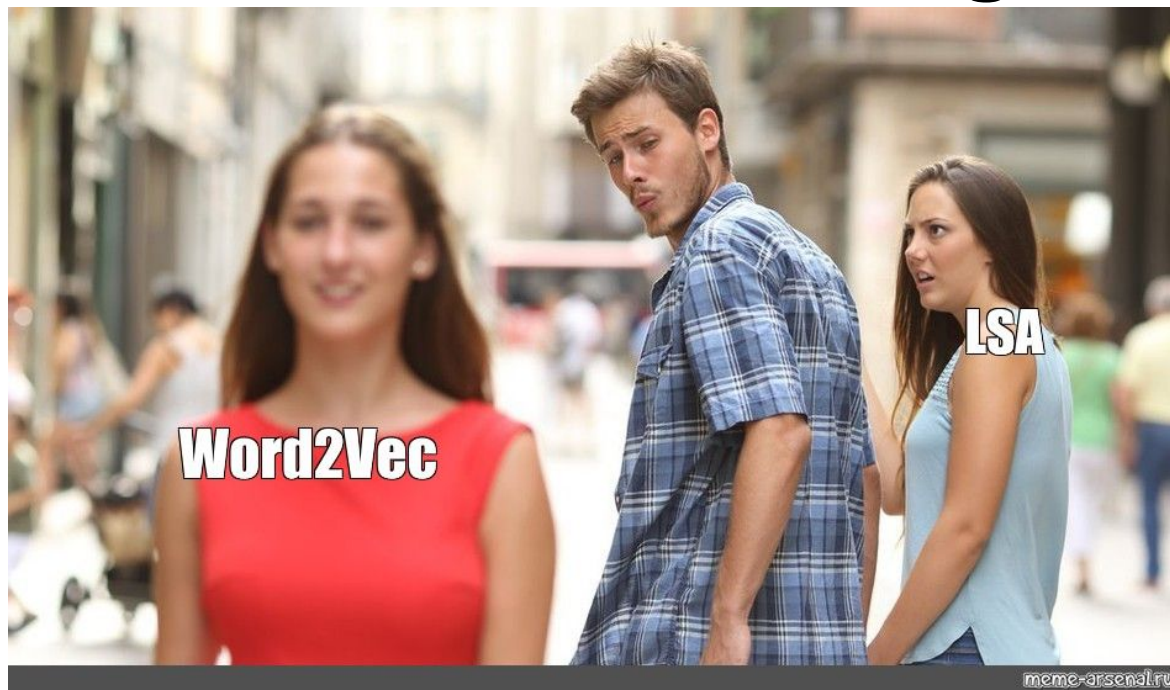
Training Time:



Test Time:

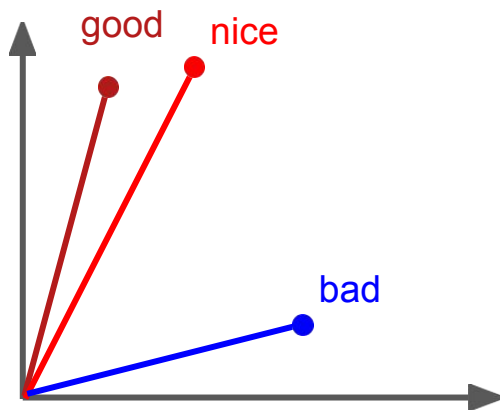
Use all neurons, but scale
activations by p

4. Word Embeddings



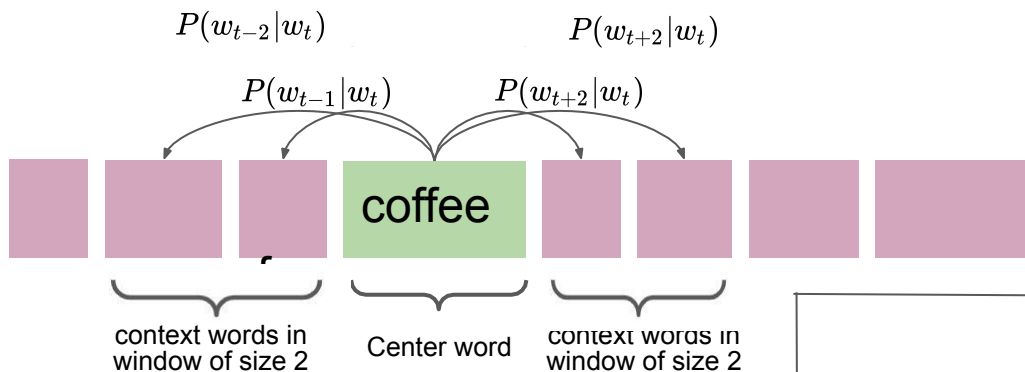
Word Embeddings

Motivated by semantic similarity

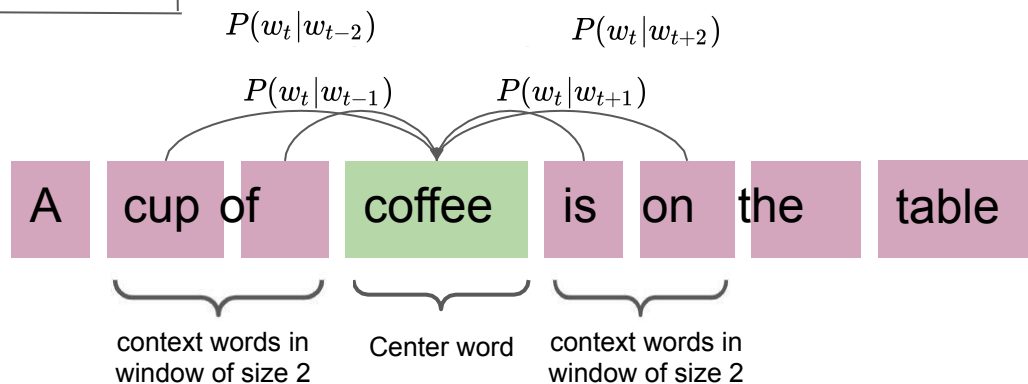


Word2Vec

Skipgram - Predict context from target



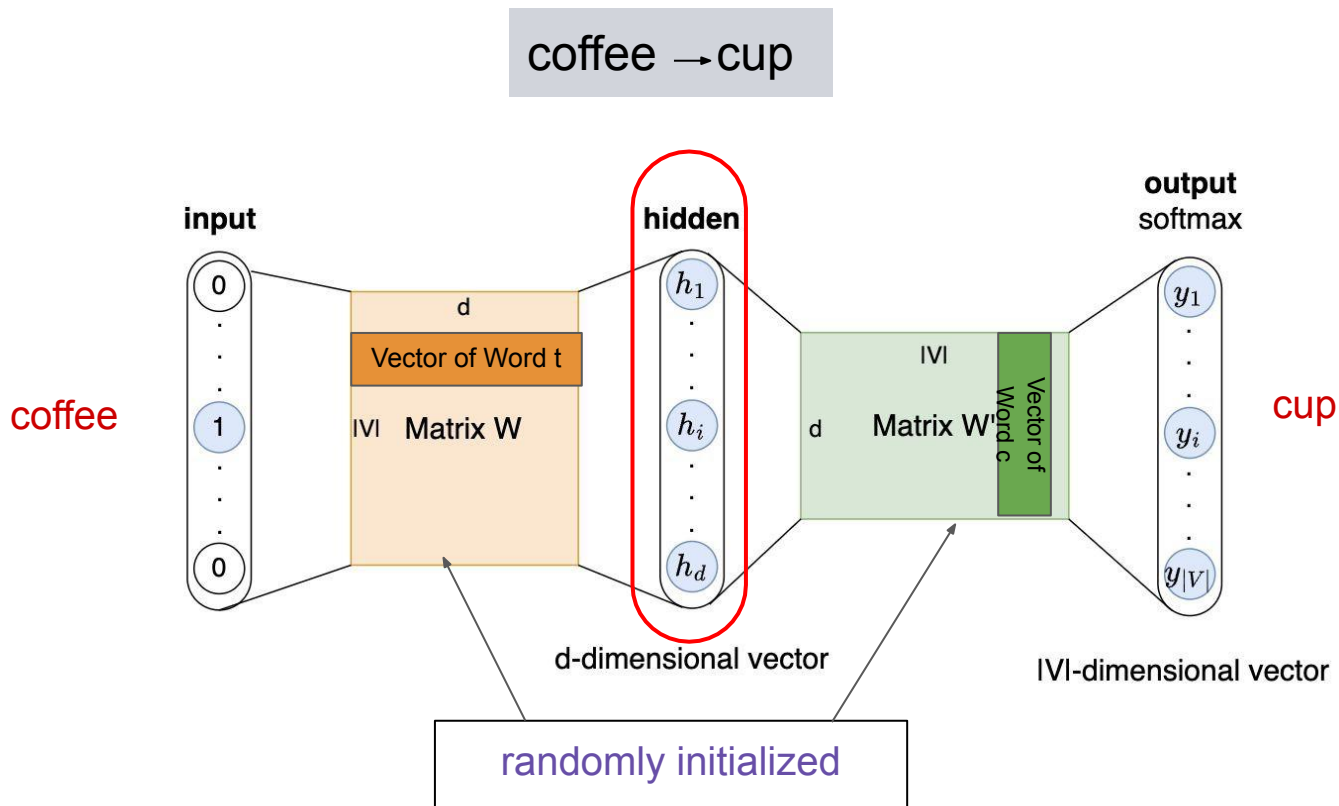
Continuous Bag of Words (CBOW) - predict target from context



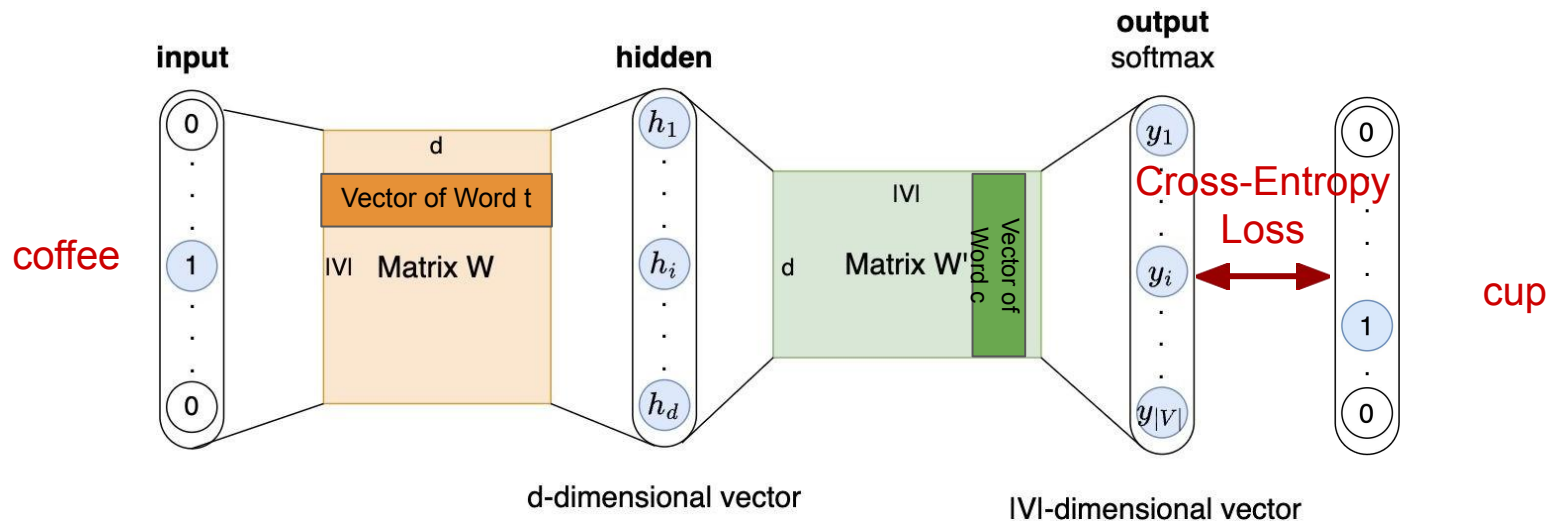
Training Data

One-hot encoded vectors of all the words in your vocabulary

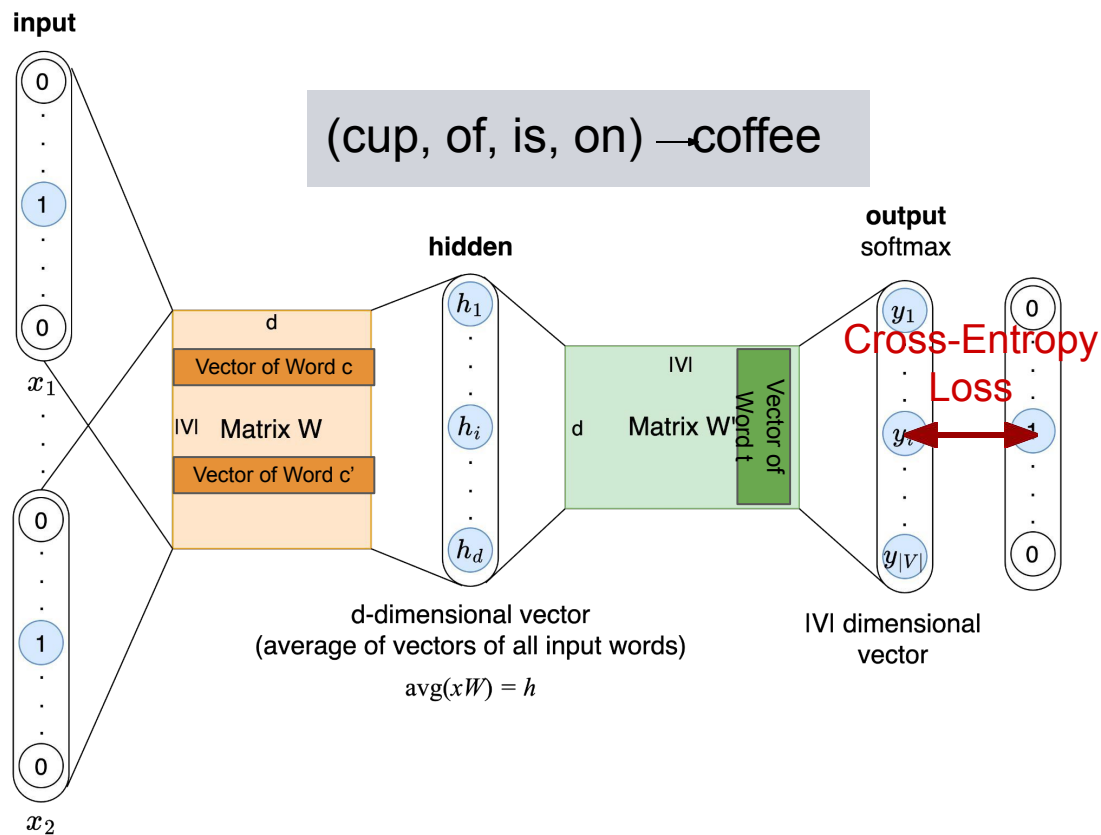
SkipGram



Calculate Loss and BackPropagate till Minima

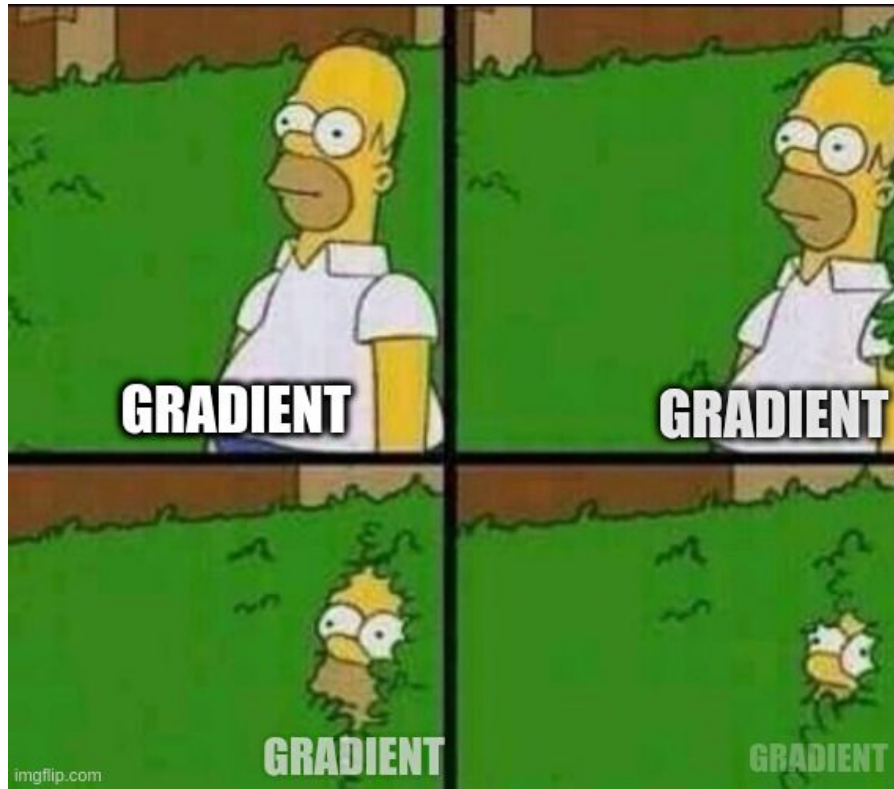


CBOW



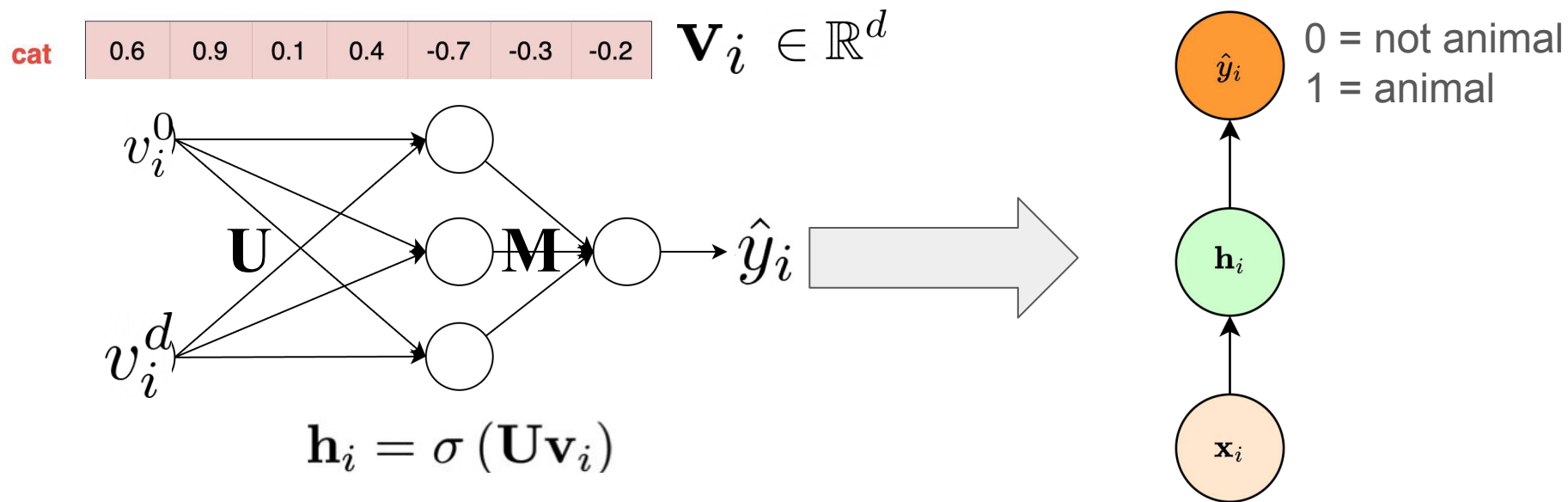
Calculate Loss
and
BackPropagate
till Minima

5. RNNs & LSTMs

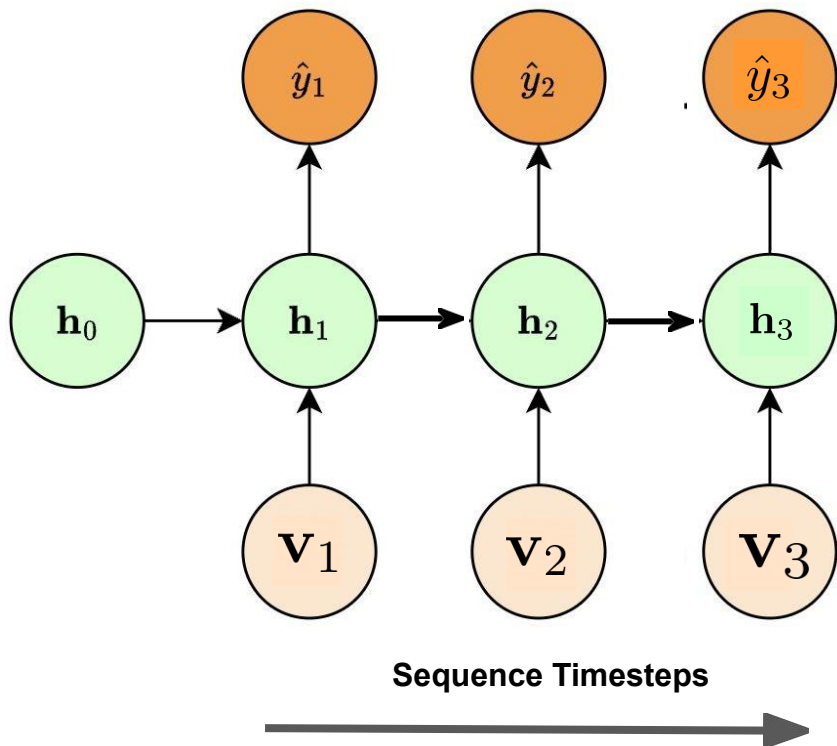


How to use word vectors with neural networks?

Let's simplify: what if we have a single word and a single output?



Recurrent neural network (RNN)



Idea: use the same parameters ($\mathbf{U}$, $\mathbf{W}$, $\mathbf{M}$) across different timesteps

Output

$$\hat{y}_i = \mathbf{M} \mathbf{h}_i$$

Hidden State

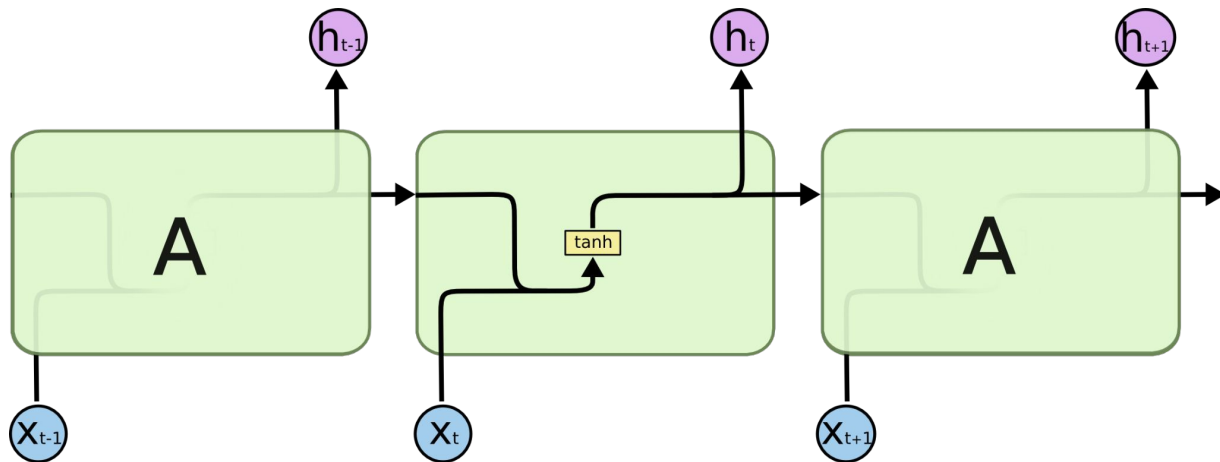
$$\mathbf{h}_i = \sigma(\mathbf{U} \mathbf{v}_i + \mathbf{W} \mathbf{h}_{i-1})$$

Word
Embeddings

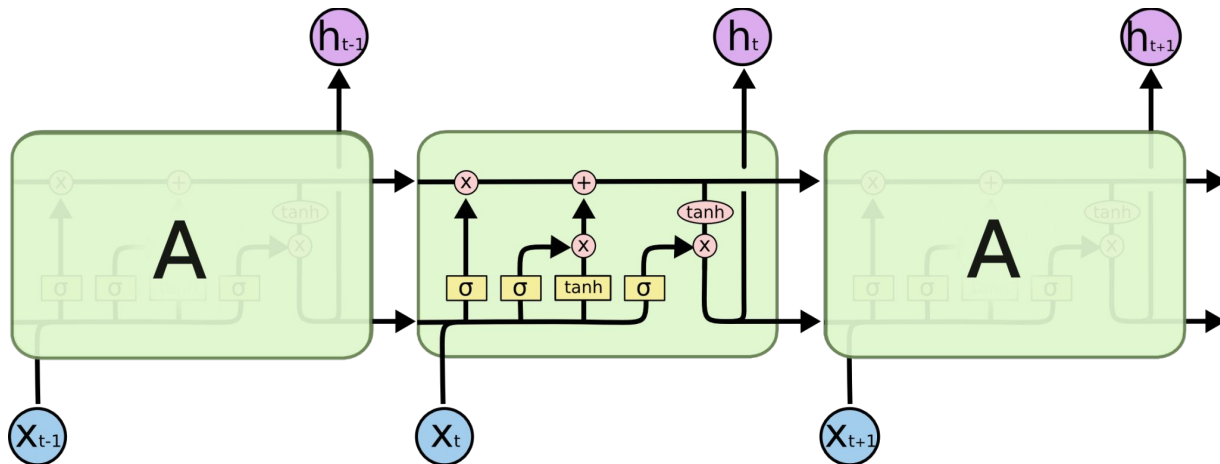
Problems with RNNs

- Issues under **loooooooooong** context
- Vanishing/exploding gradients
- Sequential - slow

RNN



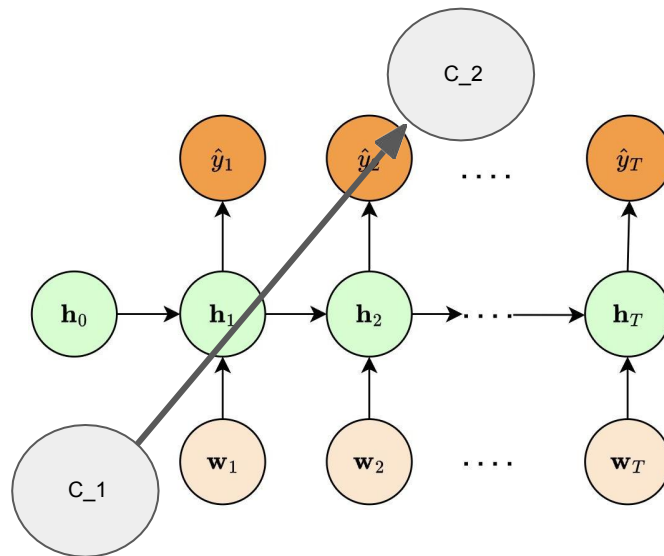
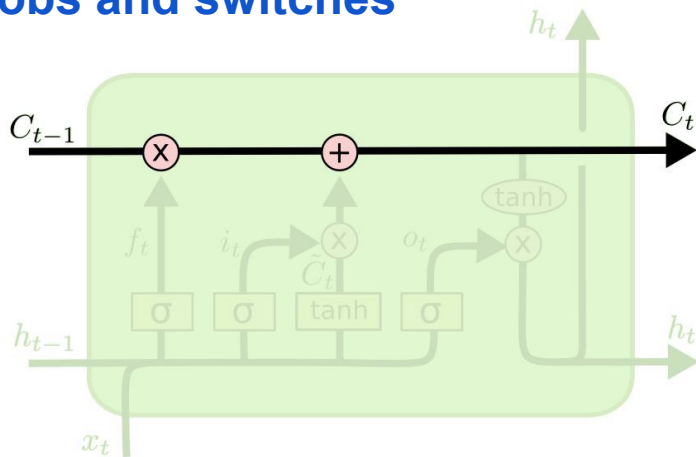
Long-short Term Memory (LSTM)



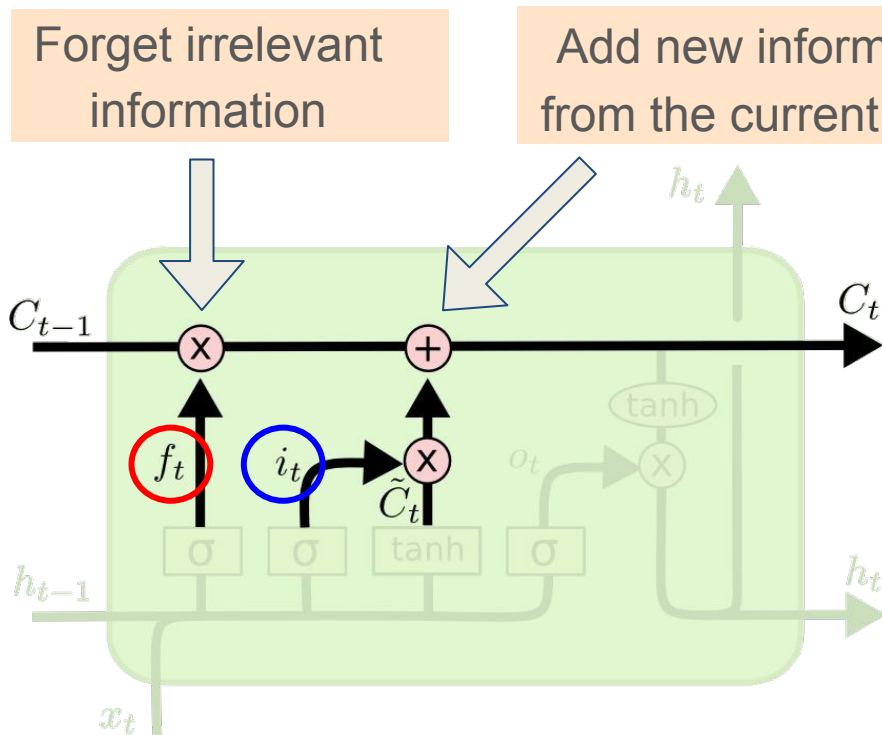
Long-short Term Memory (LSTM)

- Main idea: add a “cell” state that allows information to flow easily
 - Similar to residual connections
 - No repeated matrix multiplications!

Control flow by adding
knobs and switches



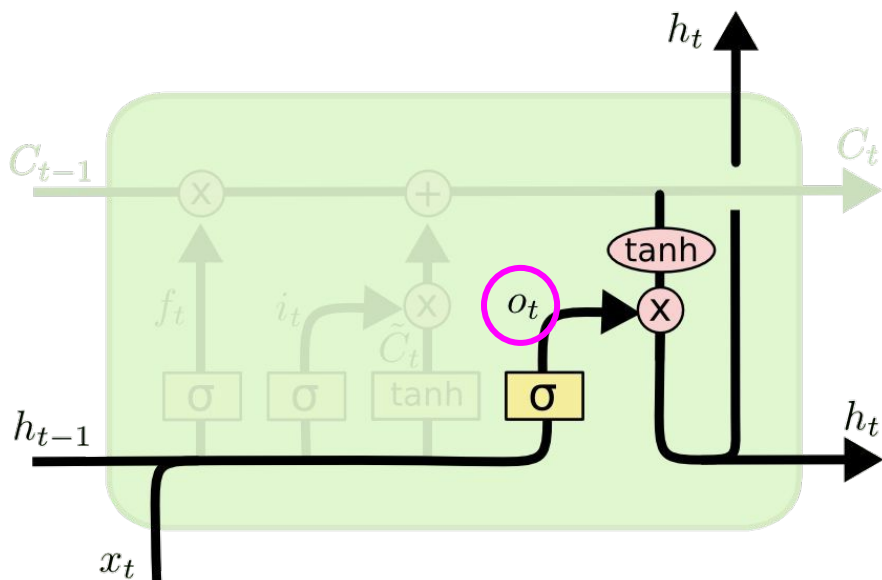
LSTM: Cell Update



Forget Gate: Controls what should be remembered in the cell state

Input Gate: Controls what information to write to the cell state

LSTM: Output Gate



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Output Gate: Controls flow of information from cell state to hidden state

LSTMs

- Perform better with long sequences
- But still sequential!!

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

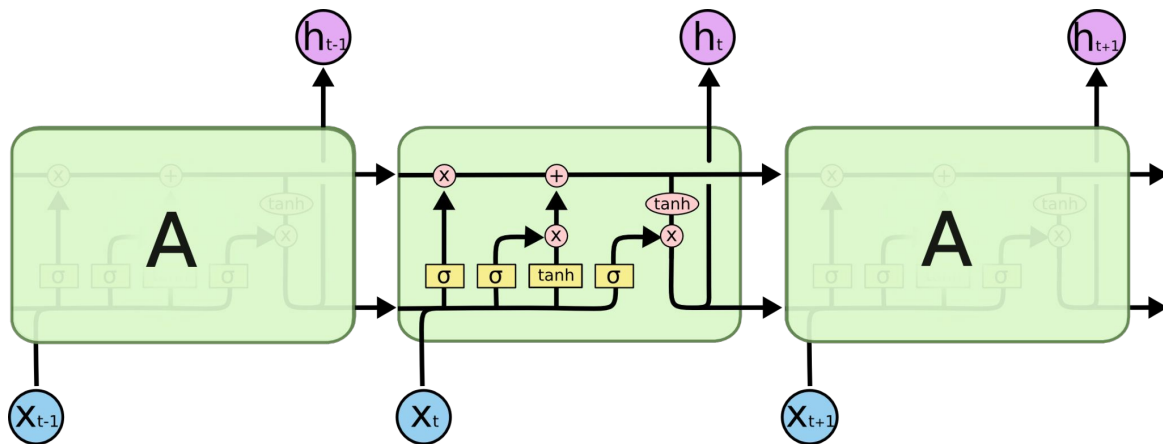
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

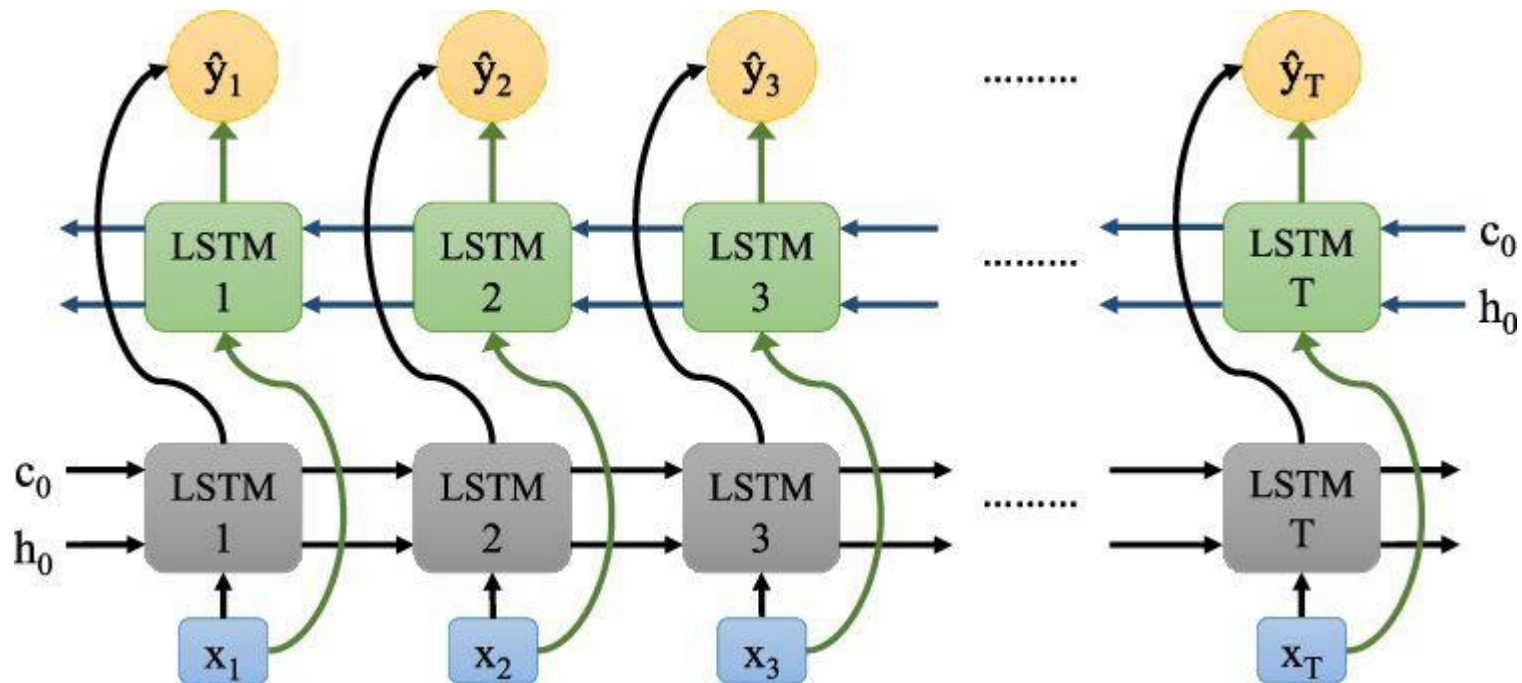
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

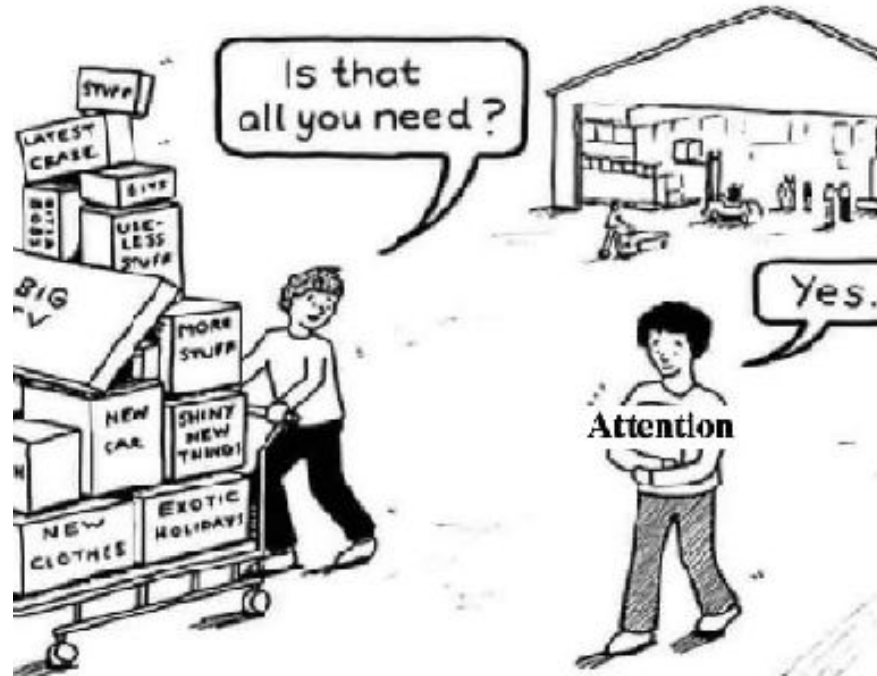
$$h_t = o_t * \tanh(C_t)$$



Bidirectional LSTM



6. Transformers & Attention



RNNs and LSTMs were sequential - Transformers are parallelized!

Token Position

Integer	0	1	2	3	4	5	6	7
Binary	0	0	0	0	1	1	1	1
	0	0	1	1	0	0	1	1
	0	1	0	1	0	1	0	1

Positional Embeddings

For a position pos and embedding dimension i :

- $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$
- $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

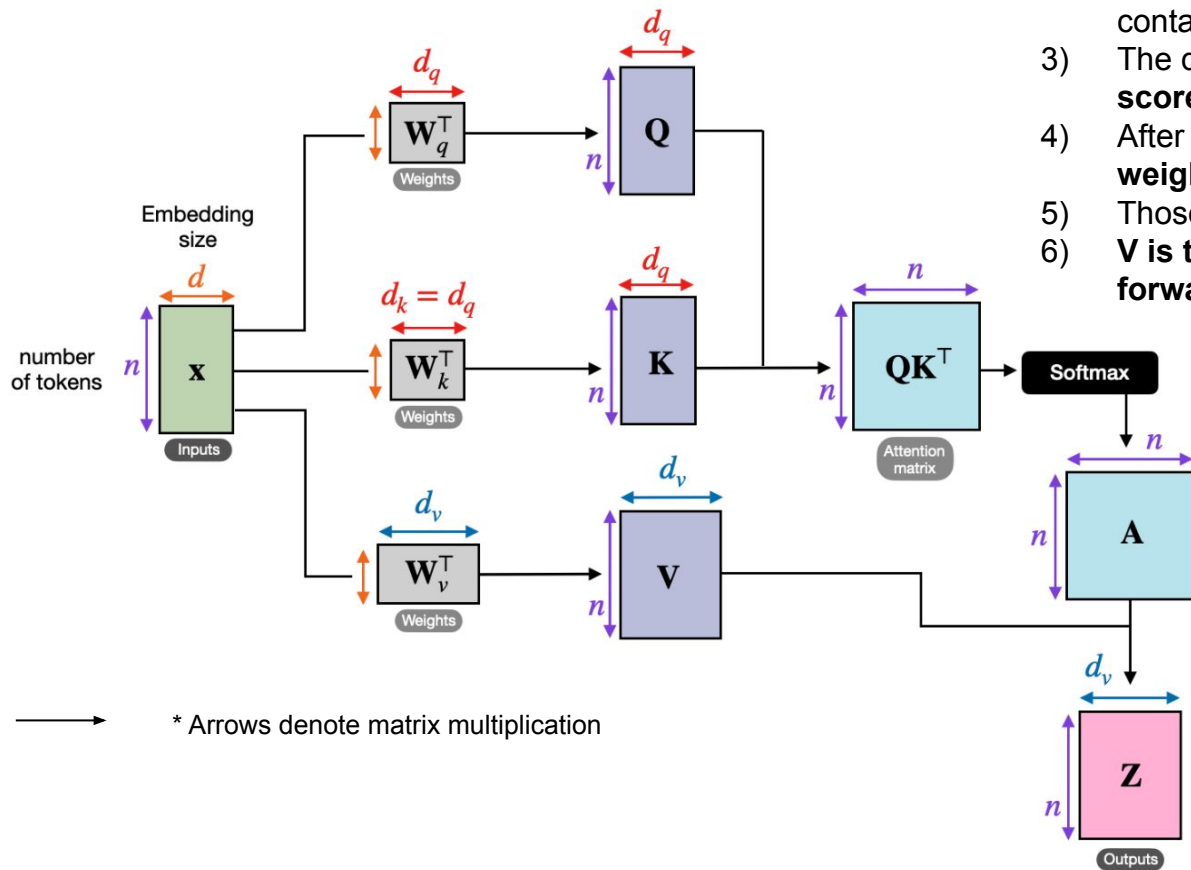
Token Position

Position	0	1	2	3	4	5	6	7
Sinusoid	0.5	0.3	0.3	0.5	0.7	0.9	0.9	0.7
	0.5	0.5	0.7	0.7	0.5	0.5	0.7	0.7
	0	1	0	1	0	1	0	1

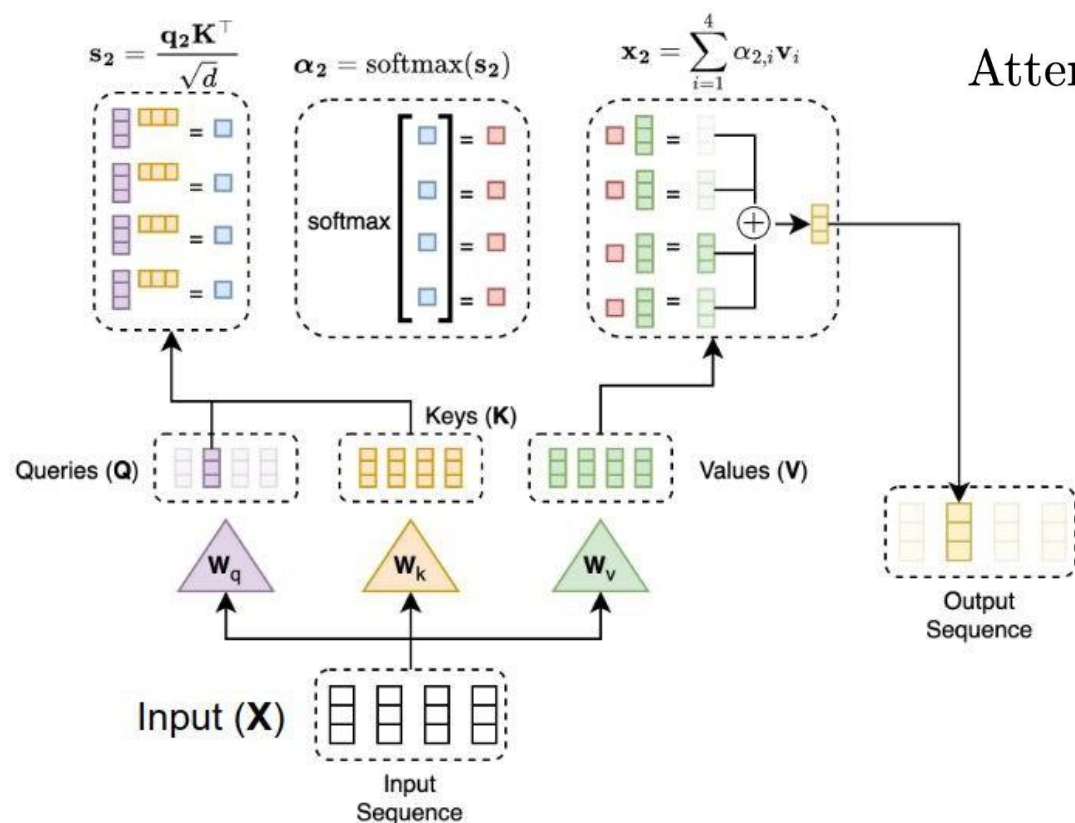
Positional Embeddings

Unmasked Self-Attention

- 1) **Queries (Q)** ask: “What am I looking for?”
- 2) **Keys (K)** say: “What kind of information do I contain?”
- 3) The dot products QK^T produce **similarity scores**
- 4) After softmax, those scores become **attention weights**
- 5) Those weights are applied to **Values (V)**
- 6) **V is the content that gets mixed and passed forward** after attention decides what matters.



Unmasked Self-Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Unmasked vs Masked Self-Attention Visualization

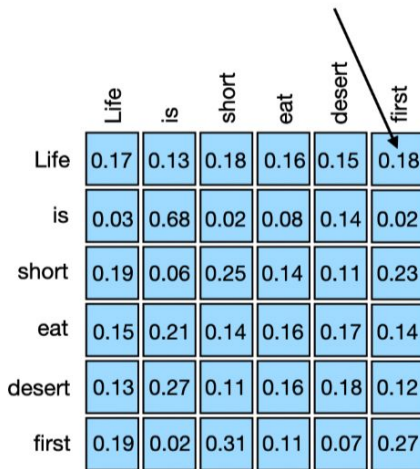
Row (Query) = the token doing the attending

Column (Key) = the token it is allowed to look at

Light blue means **allowed**

shaded means **not allowed**

Attention weight between the tokens
corresponding to "Life" and "short"

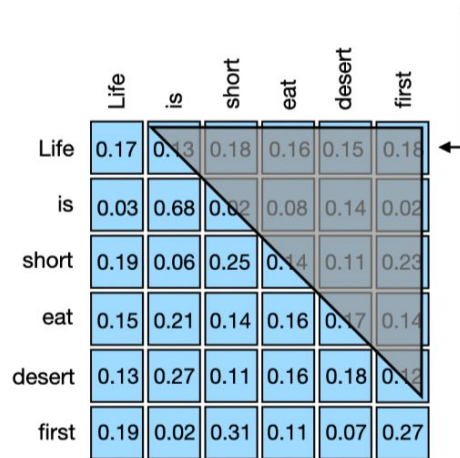


The Unmasked self-attention visualization shows a 6x6 matrix of attention weights between tokens. All cells are light blue, indicating that every token is allowed to attend to every other token. An arrow points from the text 'Attention weight between the tokens corresponding to "Life" and "short"' to the cell at row 'Life' and column 'short'.

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Unmasked

In the row for corresponding to "Life",
mask out all words that come after "Life"



The Masked self-attention visualization shows the same 6x6 matrix of attention weights, but with a mask applied. A diagonal line from the top-right to the bottom-left indicates that tokens can only attend to themselves or tokens that appear before them in the sequence. In the 'Life' row, the cells for 'is', 'short', 'eat', 'desert', and 'first' are shaded grey, indicating they are not allowed to be attended to. An arrow points from the text 'In the row for corresponding to "Life", mask out all words that come after "Life"' to the 'is' cell in the 'Life' row.

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Masked

Cross-Attention: The Target *Attends* to The Source

For example, when translating "I like eating ice cream" into another language, cross-attention helps the system understand that "ice cream" is important and should be translated correctly.

Useful in...

- Translation tasks
- Question-Answer systems
- More!

The output sequence needs to know the context of the input sequence!

The mechanics of cross-attention are not so daunting

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

It's the same thing as self-attention!

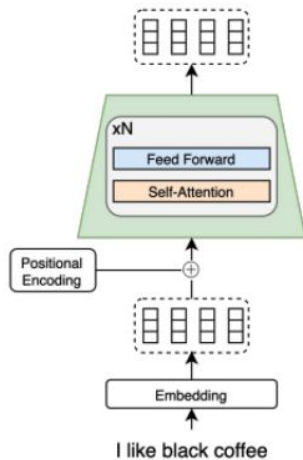
What changes:

- Queries come from the output
- Keys and Values from the input

Transformers

BERT

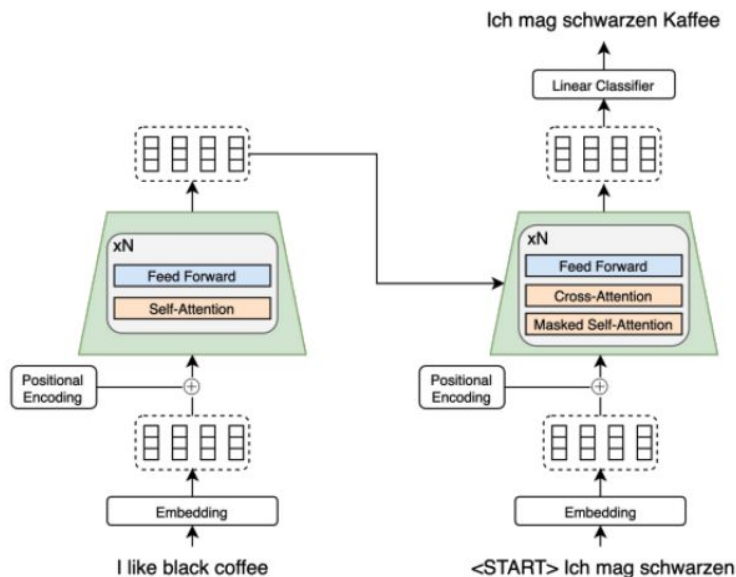
encoder only



[Devlin et al. 2019]

T5/BART

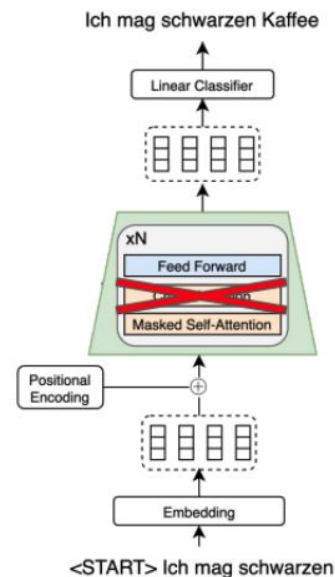
encoder-decoder model



[Raffel et al. 2020] [Lewis et al. 2020]

GPT

decoder only

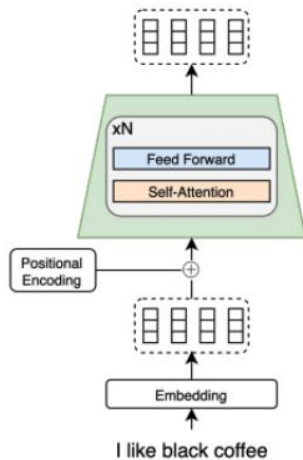


[Radford et al. 2018]

Transformers

BERT

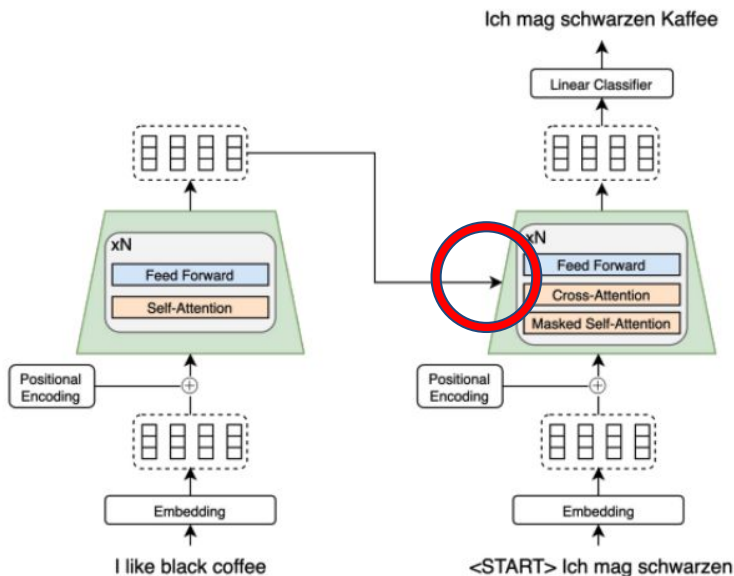
encoder only



[Devlin et al. 2019]

T5/BART

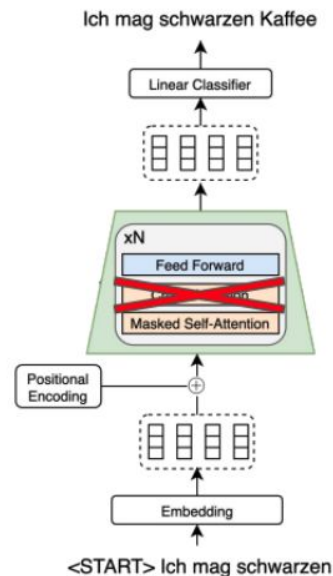
encoder-decoder model



[Raffel et al. 2020] [Lewis et al. 2020]

GPT

decoder only

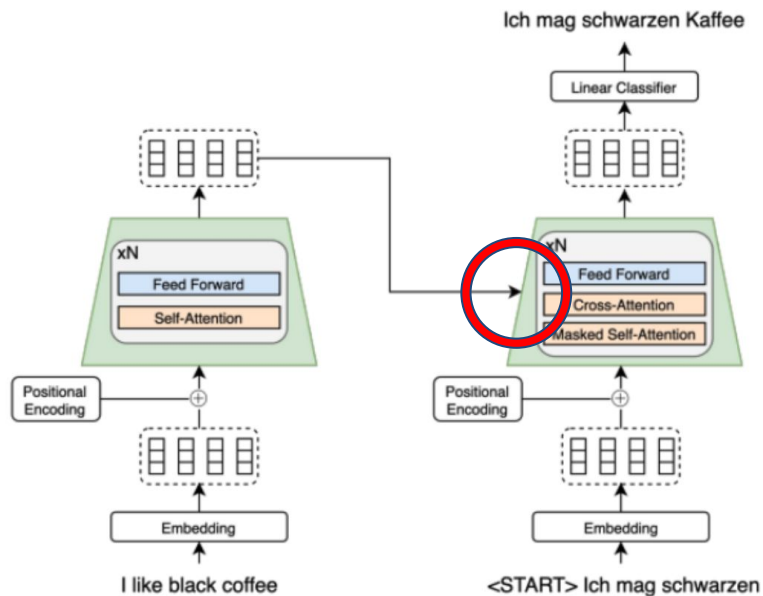


[Radford et al. 2018]

Transformers

T5/BART

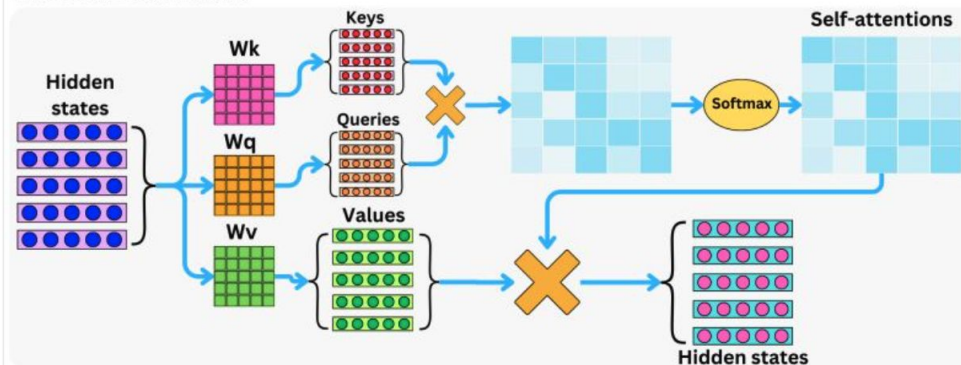
encoder-decoder model



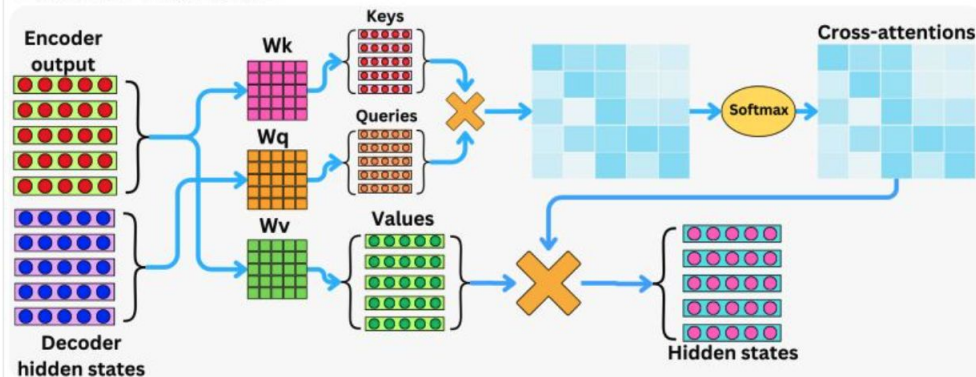
[Raffel et al. 2020] [Lewis et al. 2020]

Self-Attention VS Cross-Attention TheAiEdge.io

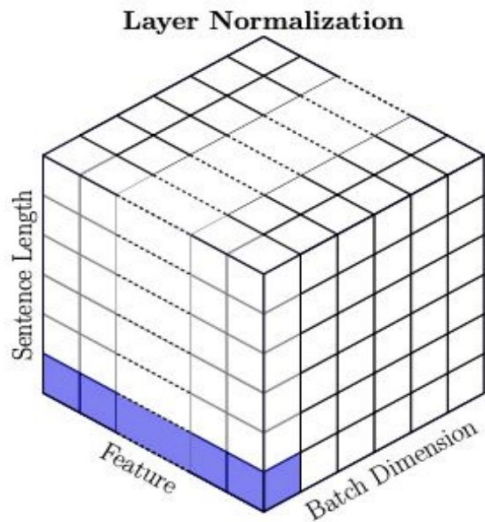
The Self-Attentions



The Cross-Attentions



One last attribute - LayerNorm



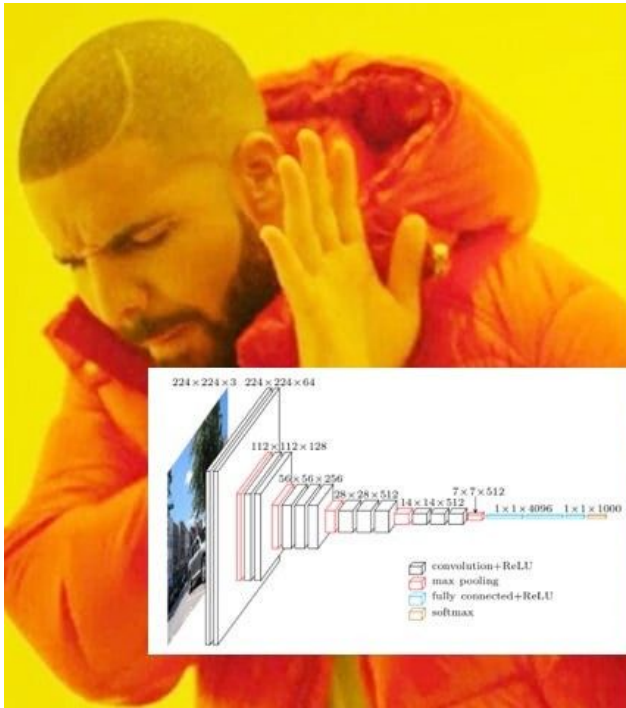
- Normalize the features for each token... individually.
- Mechanics are the same as BatchNorm, but normalize each token *separately*.

Leads to:

- Stable gradients
- Faster training
- etc.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

9. CNNs & U-Nets

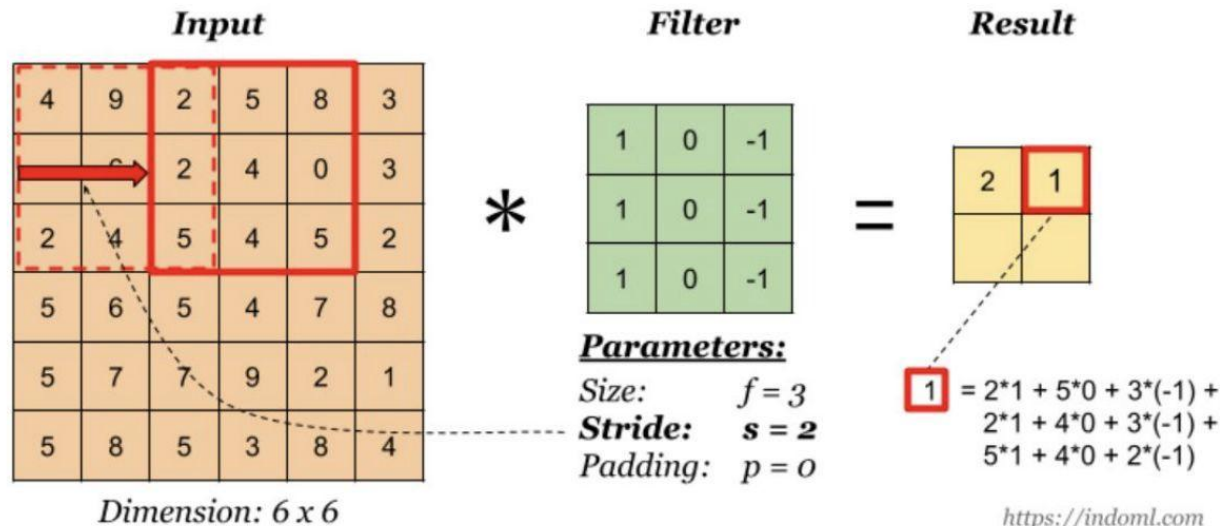


CNN review

HW1 Q4

Output dimension:

$$(\lfloor \frac{n - k + 2p}{s} \rfloor + 1) \times (\lfloor \frac{n - k + 2p}{s} \rfloor + 1) \times l$$



Filter with stride (s) = 2

CNN

Useful formula

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{in} : number of input features

n_{out} : number of output features

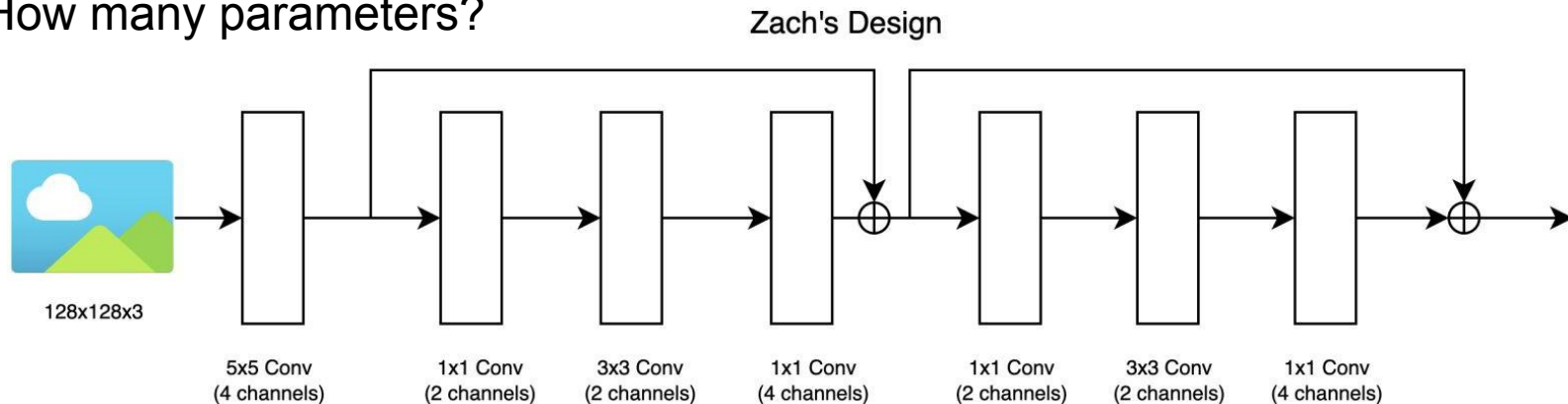
k : convolution kernel size

p : convolution padding size

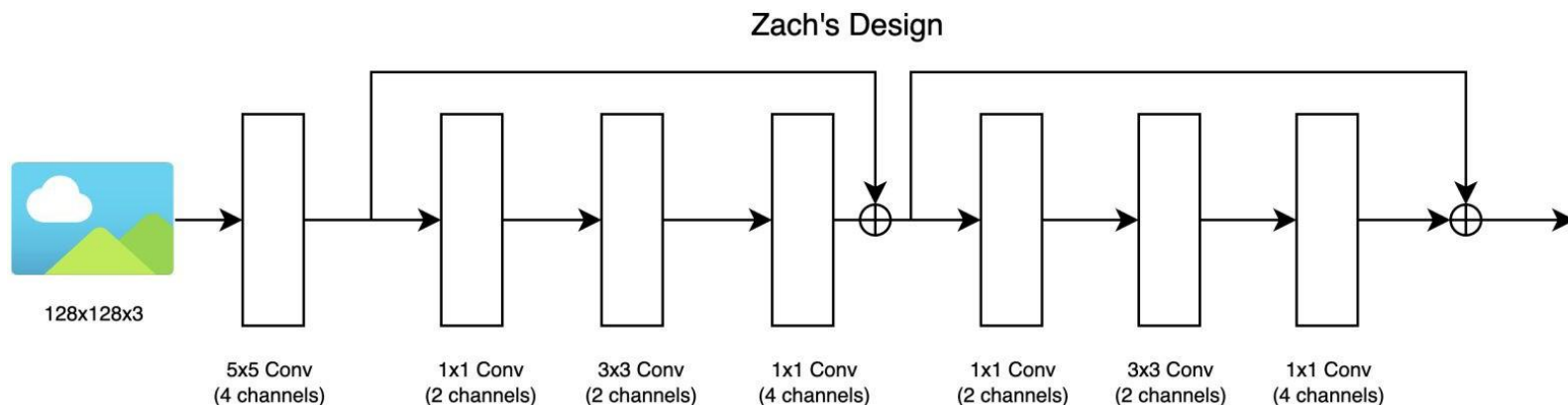
s : convolution stride size

SP24 Prelim Q7

How many parameters?



SP24 Prelim Q7



Layer 1: $(5 \times 5) \times 3 \times 4 = 300$

Layer 2: $(1 \times 1) \times 4 \times 2 = 8$

Layer 3: $(3 \times 3) \times 2 \times 2 = 36$

Layer 4: $(1 \times 1) \times 2 \times 4 = 8$

Layer 5: $(1 \times 1) \times 4 \times 2 = 8$

Layer 6: $(3 \times 3) \times 2 \times 2 = 36$

Layer 7: $(1 \times 1) \times 2 \times 4 = 8$

Total: 404

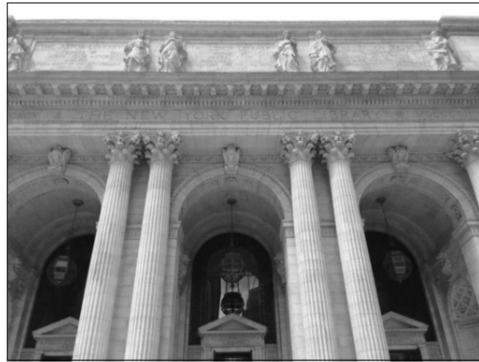
The set of filters is:

$$\left\{ \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \right\}$$

CNN review

Which output comes from which filter?

Input image:



SP24 Prelim Q5

<https://setosa.io/ev/image-kernels/>

CNN review

The set of filters is:

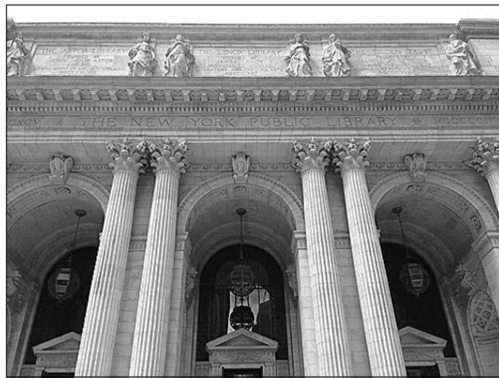
$$\left\{ \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \right\}$$

Which output comes from which filter?

Input image:



Sharpen



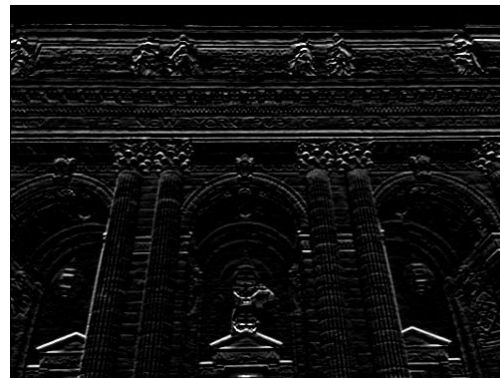
Edge



Left sobel



Bottom sobel



SP24 Prelim Q5

<https://setosa.io/ev/image-kernels/>

Brief U-Net Review

✓ Convolutions

Allow parallelization to extract latent vector for each pixel

✓ Hourglass

Improve efficiency by reducing computations with downsampling

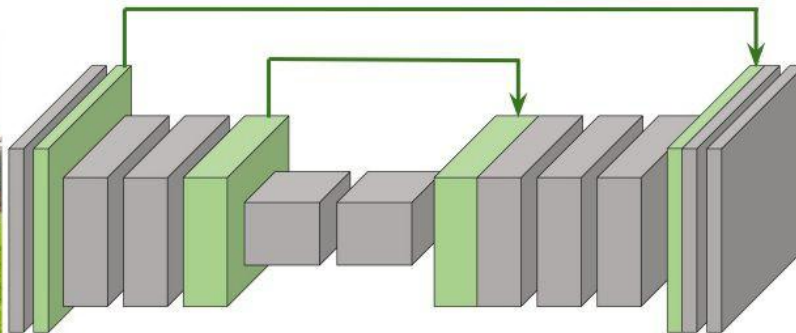
Increase receptive field size by convolving on downsampled feature maps

✓ Skip Connections

Improve prediction quality by combining low-level image features



Input Image

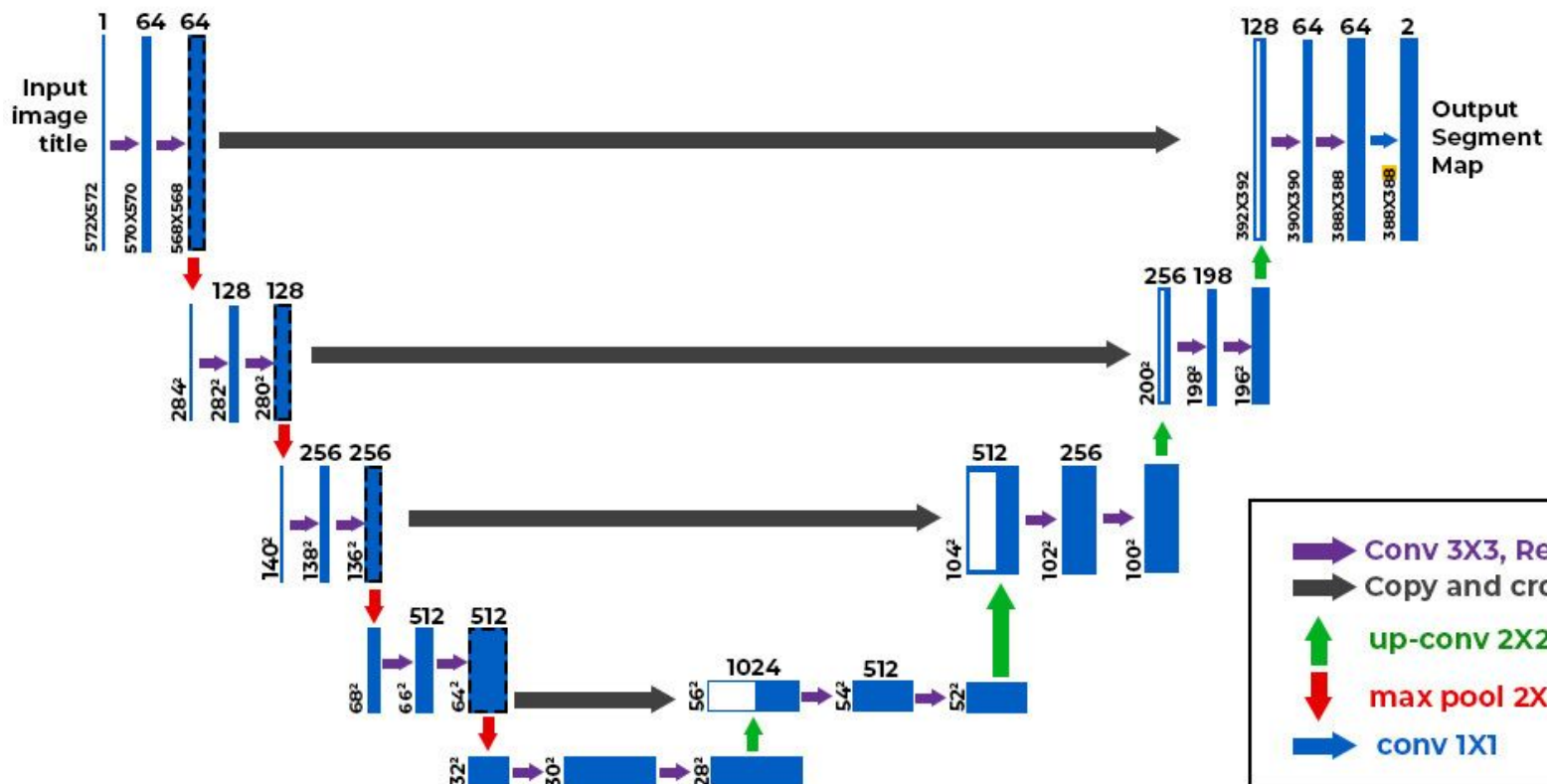


Hourglass CNN with Skip Connections



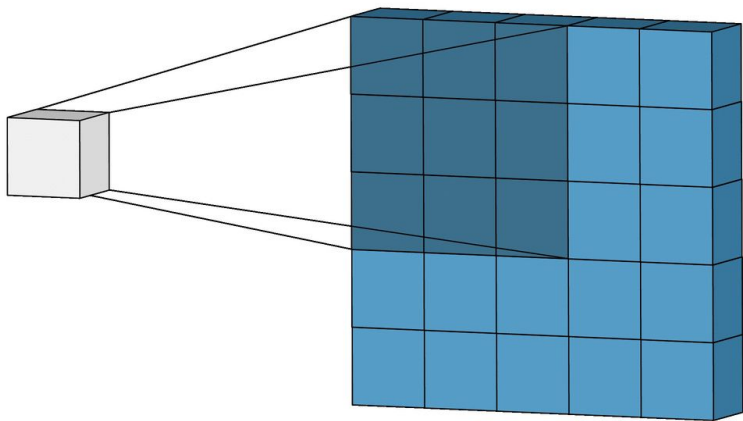
Prediction

UNet: Architecture

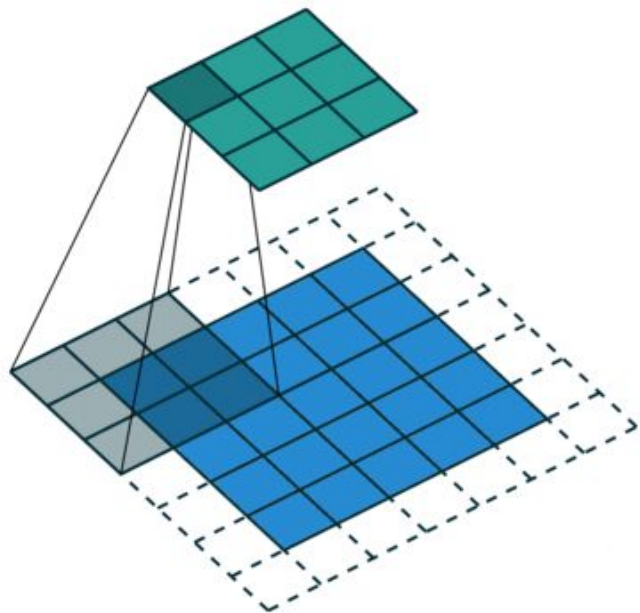


Convolution vs Deconvolution

Convolution

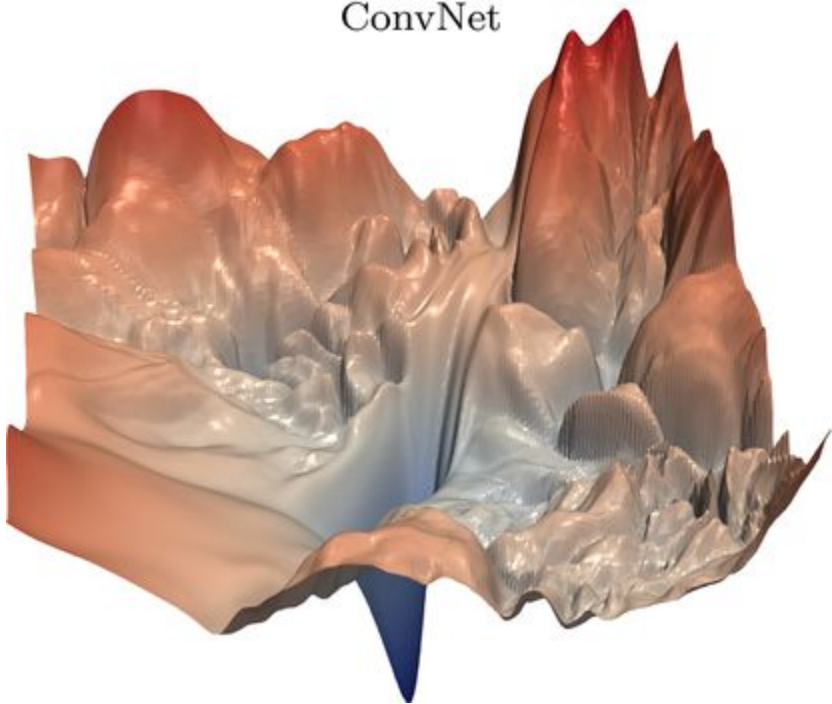


Deconvolution

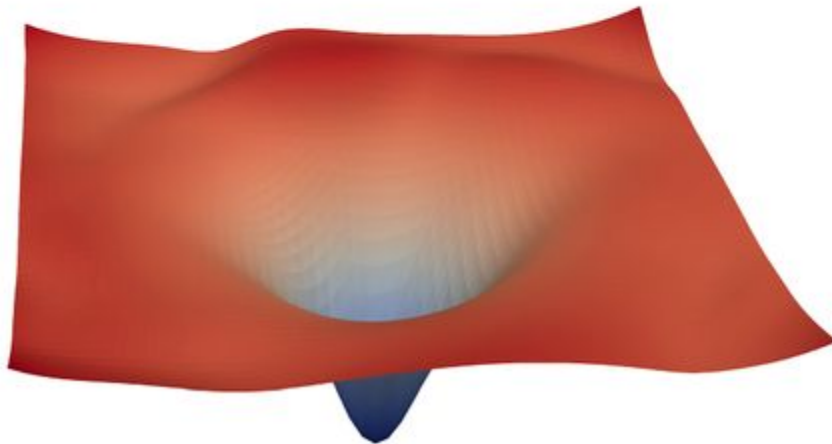


Residual Connections

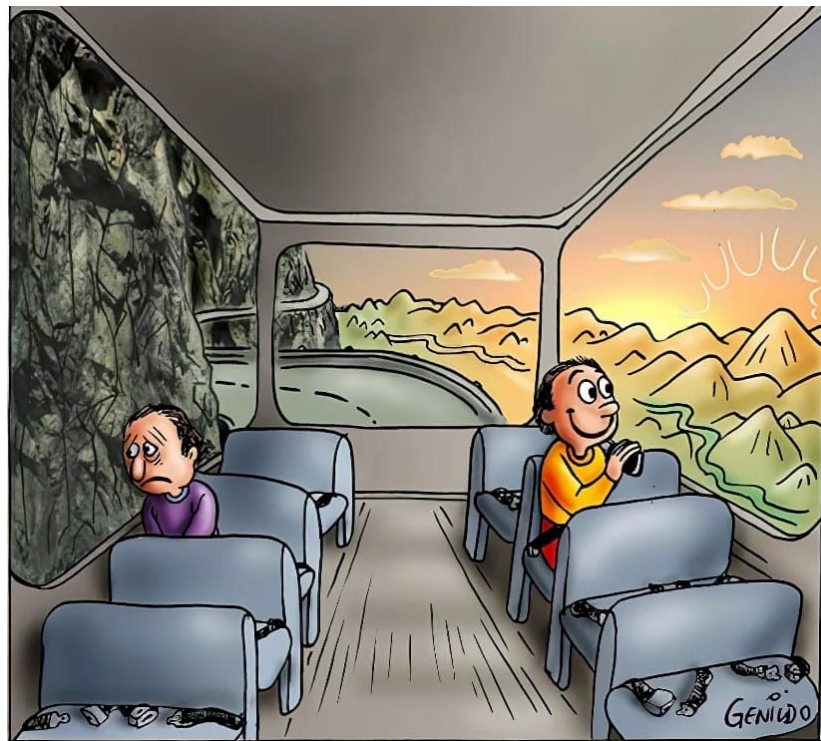
ConvNet



ResNet



7. Contrastive Learning



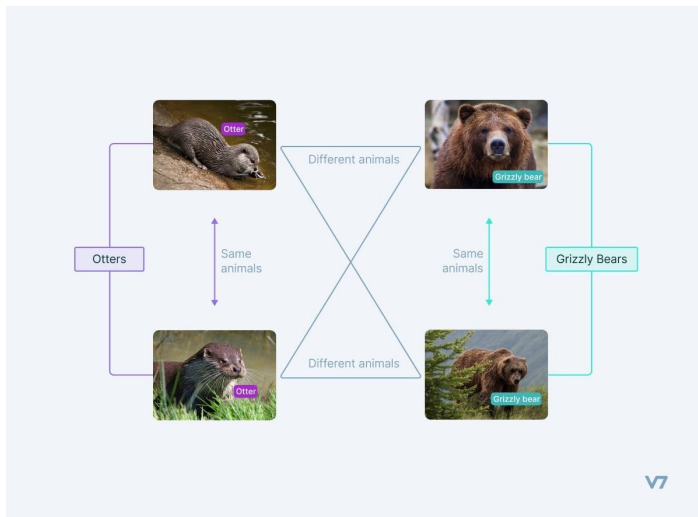
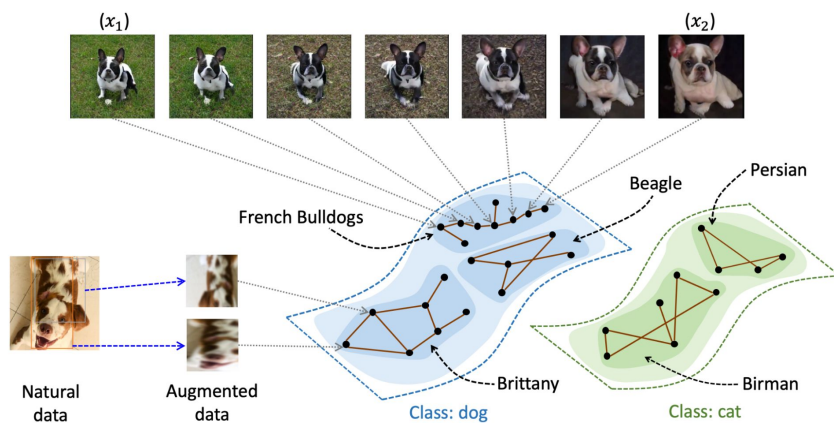
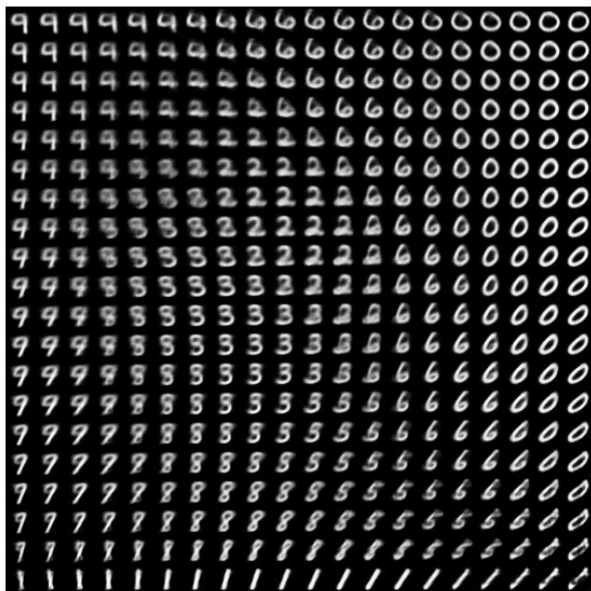
Quick thing to remember:

Contrastive learning is usually used for training **encoders** with a **meaningful latent space**

Ex:

- Image with text
- Image with audio
- Text with audio

Latent Spaces



Core intuition is that similar points should be nearby and dissimilar points should be far away!

Contrastive Learning

Q: How to get an image embedder?

→ Self-Supervised Learning

- ◆ Pre-training Task
- ◆ Contrastive Learning

$\mathbf{x}$: Sample

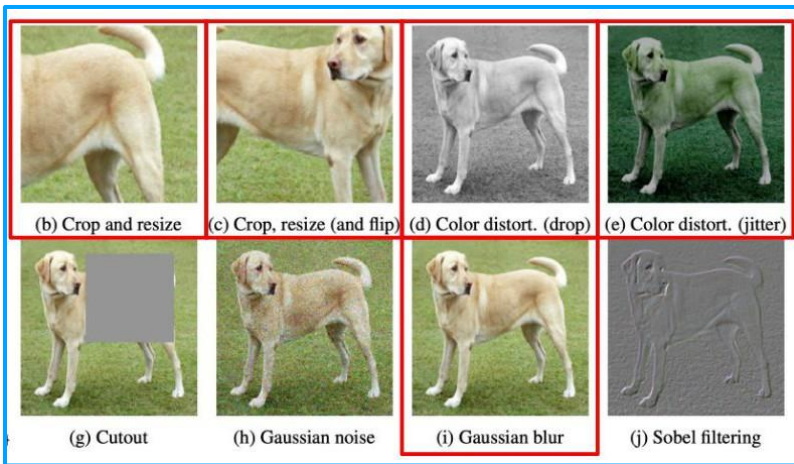


(a) Original

$\mathbf{x}^-$: **Negative** examples (*different images*)
(+negative augmentations)



$\mathbf{x}^+$: **Positive** examples (*augmentations*)



Contrastive Learning

Q: How to get an image embedder?

→ Self-Supervised Learning

- ◆ Pre-training Task
- ◆ Contrastive Learning

: Sample

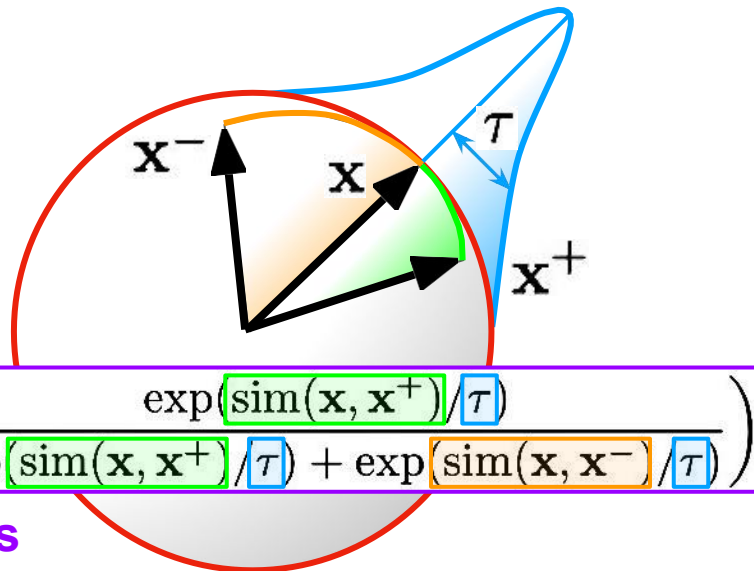
: Negative examples

$\mathbf{x}^-$: Positive examples

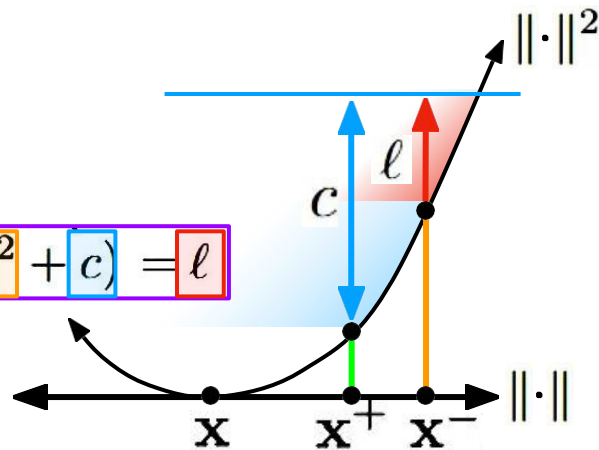
$\mathbf{x}^+$

$$\max(0, \|f(\mathbf{x}_i) - f(\mathbf{x}^+)\|^2 - \|f(\mathbf{x}_i) - f(\mathbf{x}^-)\|^2 + c) = \ell$$

Triplet Loss



SimCLR Loss



Contrastive Learning (Multimodal)

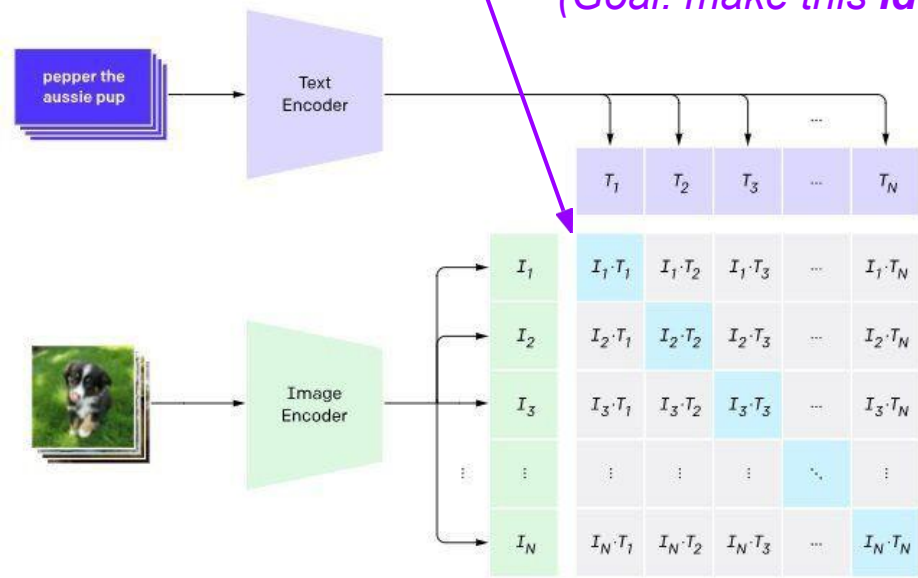
a **multimodal**

Q: How to get an ~~image~~ embedder?

→ Self-Supervised Learning

- ◆ Pre-training Task
- ◆ Contrastive Learning

1. Contrastive pre-training

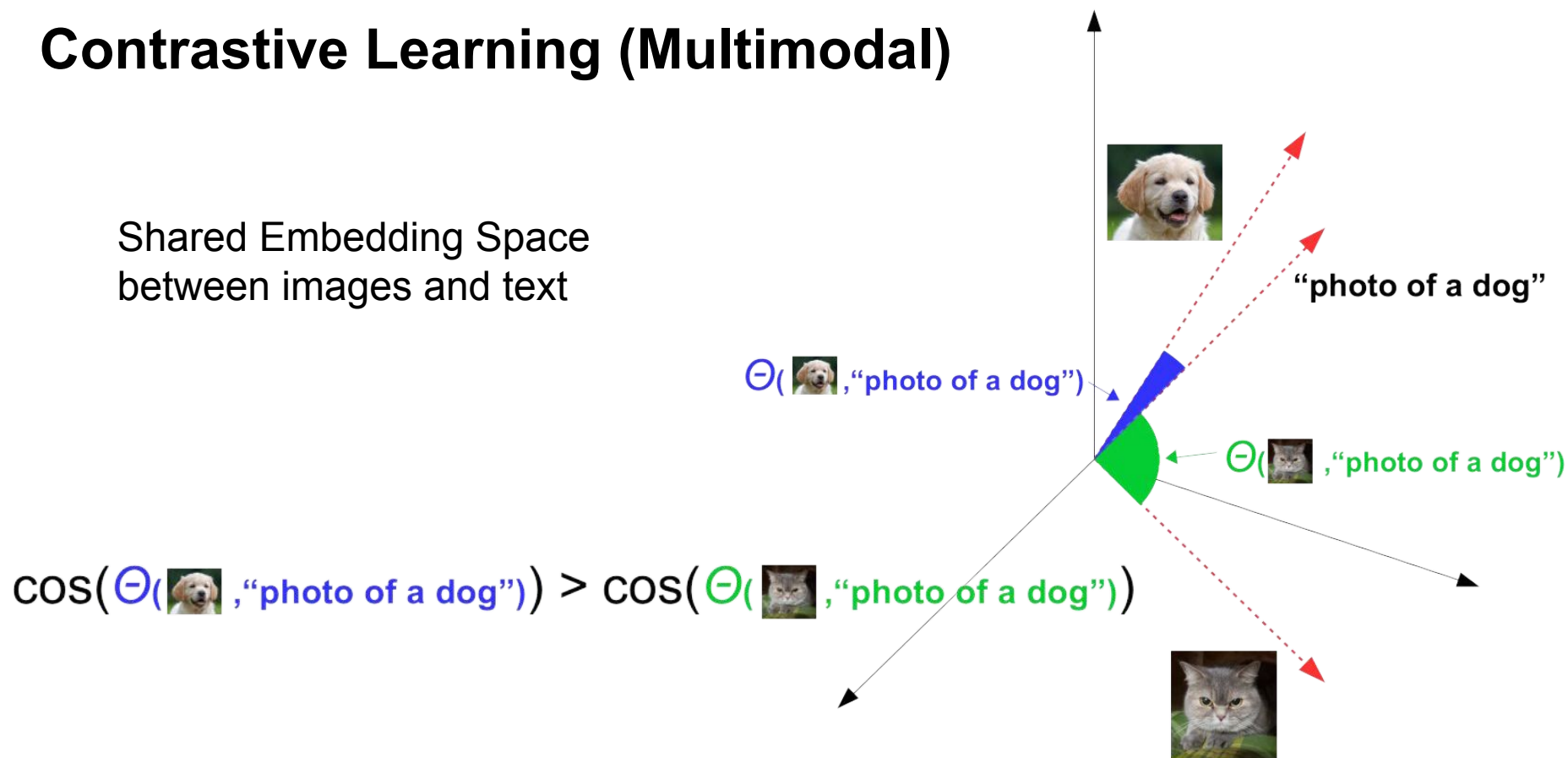


Cosine similarity between text embedding and image embedding.

(Goal: make this **Identity**)

Contrastive Learning (Multimodal)

Shared Embedding Space
between images and text



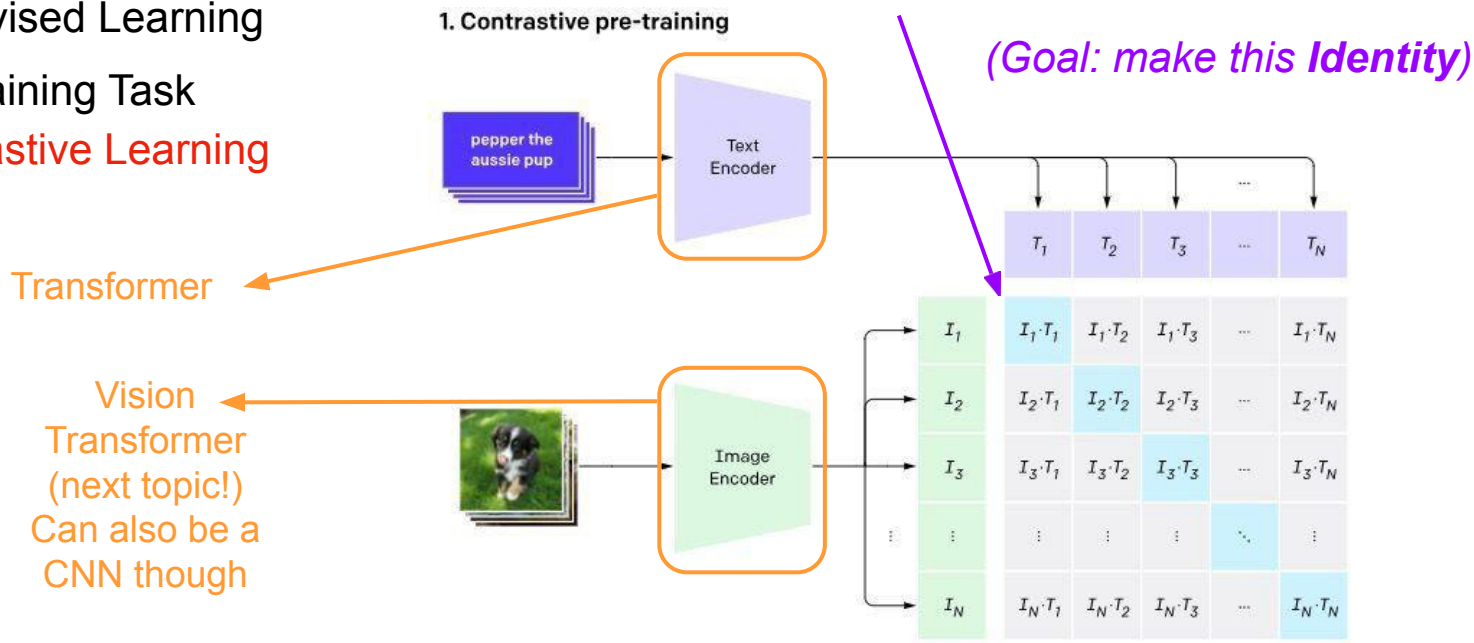
Contrastive Learning (Multimodal)

a **multimodal**

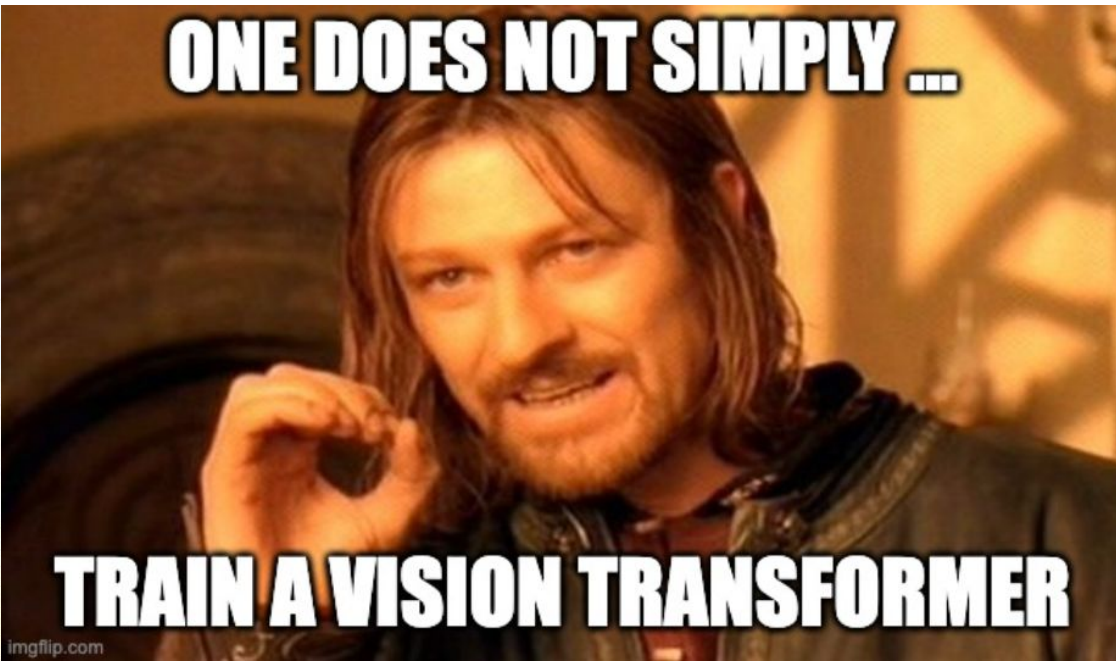
Q: How to get an ~~image~~ embedder?

→ Self-Supervised Learning

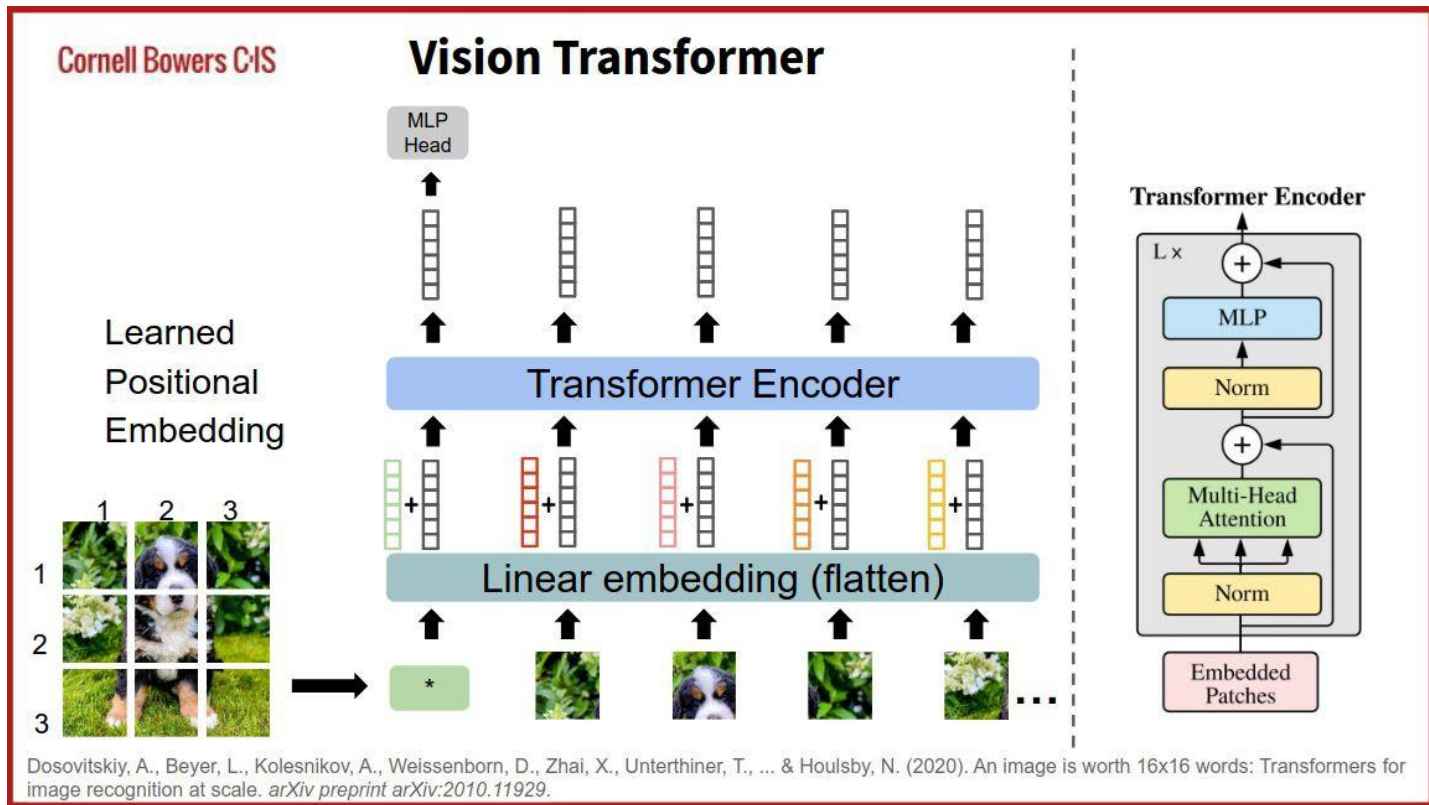
- ◆ Pre-training Task
- ◆ Contrastive Learning



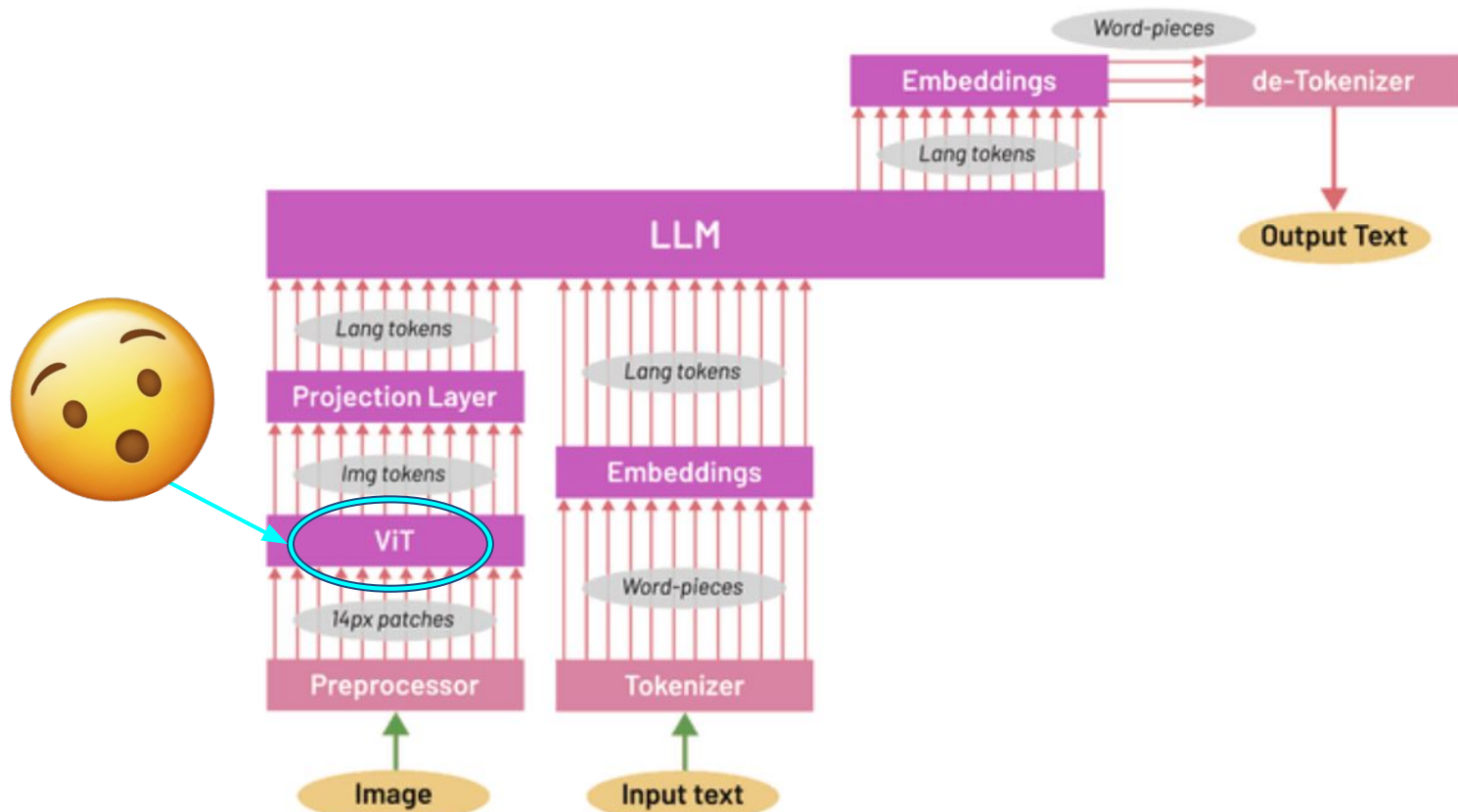
8. Vision Transformers (ViTs)



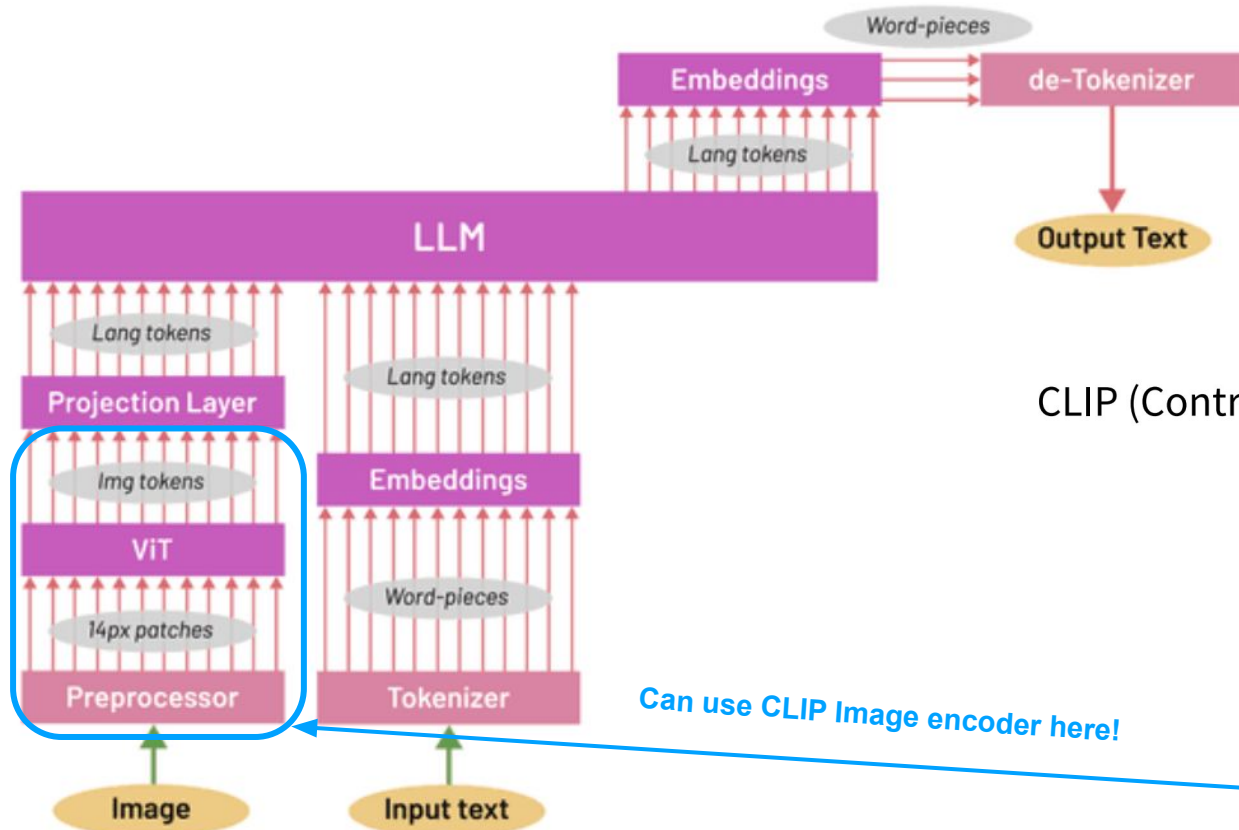
Vision Transformers (ViT)



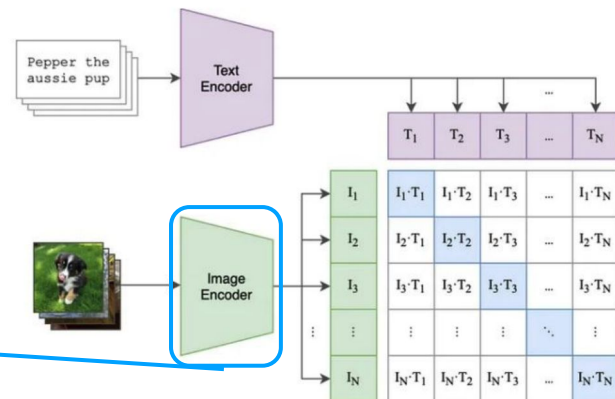
Vision Language Models



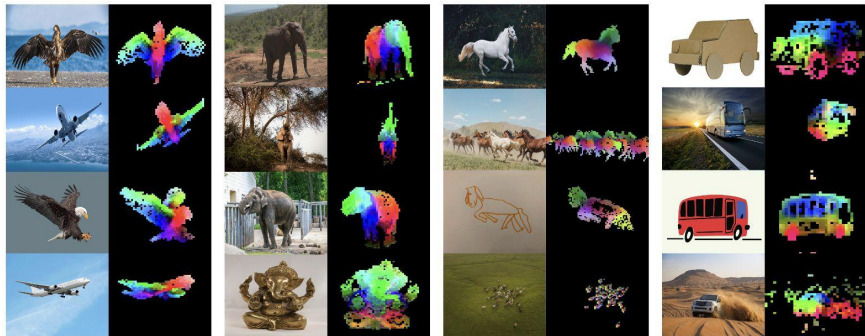
Vision Language Models



CLIP (Contrastive Language–Image Pre-training)



Vision Transformers (ViT) *(vs deep CNNs)*



class: zebra



class: convertible



ViTs *start* with a global receptive fields.
features.

- Interpretable long-range dependencies from attention are visible early on.

CNNs *take a while* to combine global

- Starts with local features and takes a while to combine them to form global representations
- In the feature maps above: yellow/red = the model thinks these regions are **important**

CNNs

- Translational invariance
- Simple, proven architecture
- Capture features at different scales
- Required less data

ViTs

- Attention mechanisms
- Allow for multimodal data integration

Swin Transformers

SWIN “Scaled Window”

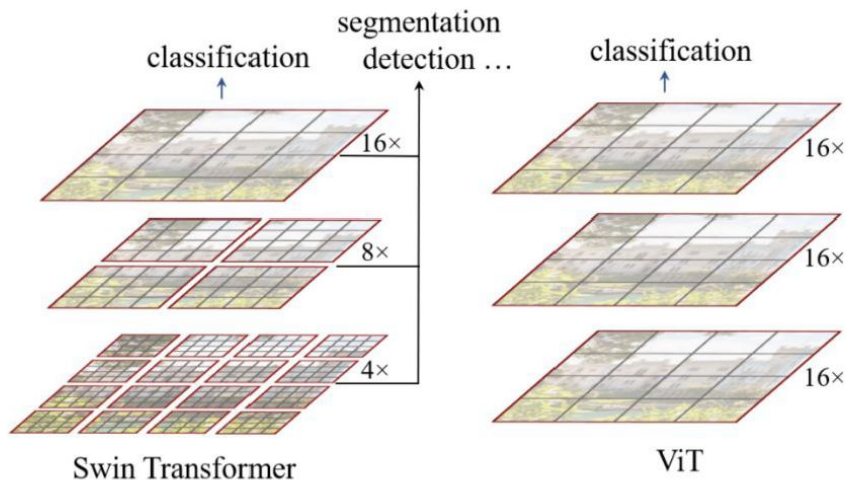
In a normal ViT:

- Every patch attends to every other patch
- Cost = 💀 quadratic in image size

In Swin:

- Image is split into small windows
- Attention happens only inside each window (a window is a collection of neighboring patches)

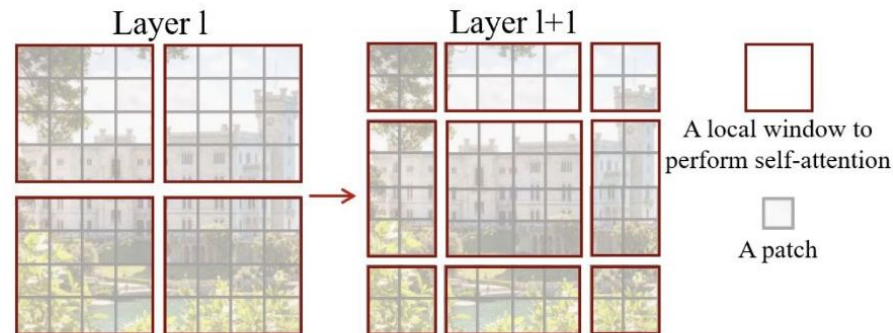
Cornell Bowers CIS



Cornell Bowers CIS

Shifted Window attention

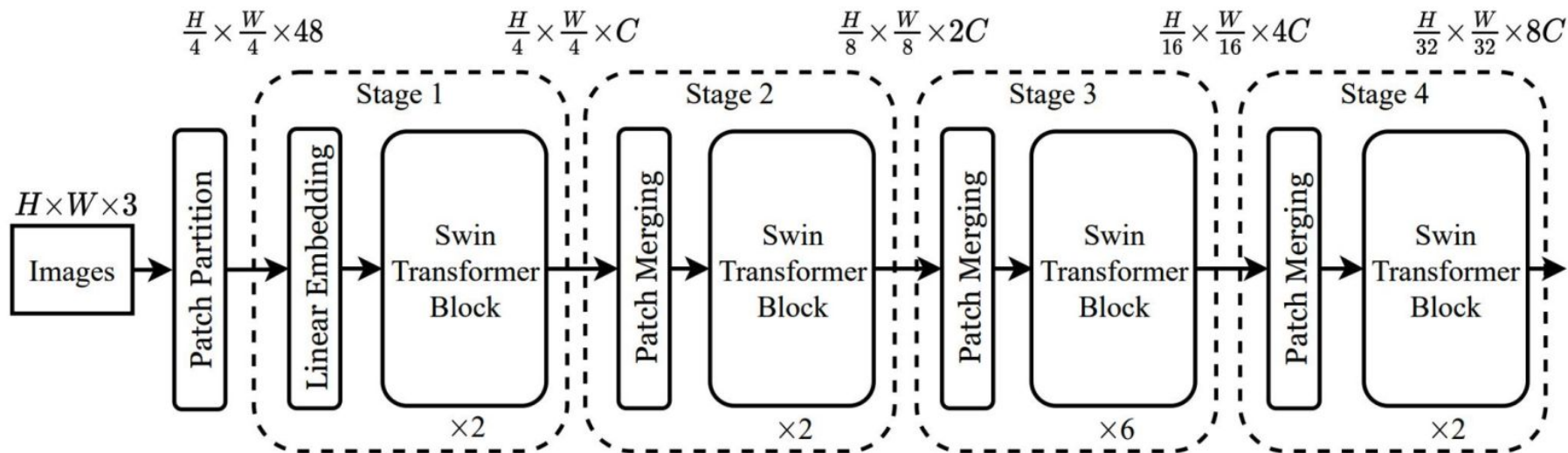
Linear computational complexity with respect to image size



Swin Transformers

SWIN “Scaled Window”

- Popular for large images and segmentation because of hierarchical features.



Practice Debugging Question

There are 4 lines of code with bugs.

- Assume `PositionalEncoding()` and `VLMModel()` class are **implemented correctly**. Assume all PyTorch tensor operations are correct (no mistakes involving operations such as `.transpose()`, `.view()`, `.reshape()`, `.size()`, etc).
- The problem assumes **number of text tokens (T) = number of image patches (P) = 16**
- There are **no crashes** when training, so **no syntax bugs**. All of the bugs are **silent**, and after training the model does not appear to have learned anything.

See code (with separate solutions) here:

https://drive.google.com/drive/folders/11Lsyy5_aynIL5c6RSLjTbx8eLvBraE_r?usp=drive_link

10. GANs



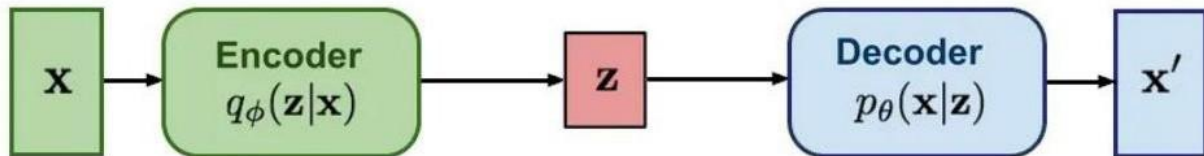
JAKE-CLARK.TUMBLR

GANs

GAN: Adversarial training



VAE: maximize variational lower bound



Diffusion models:
Gradually add Gaussian noise and then reverse



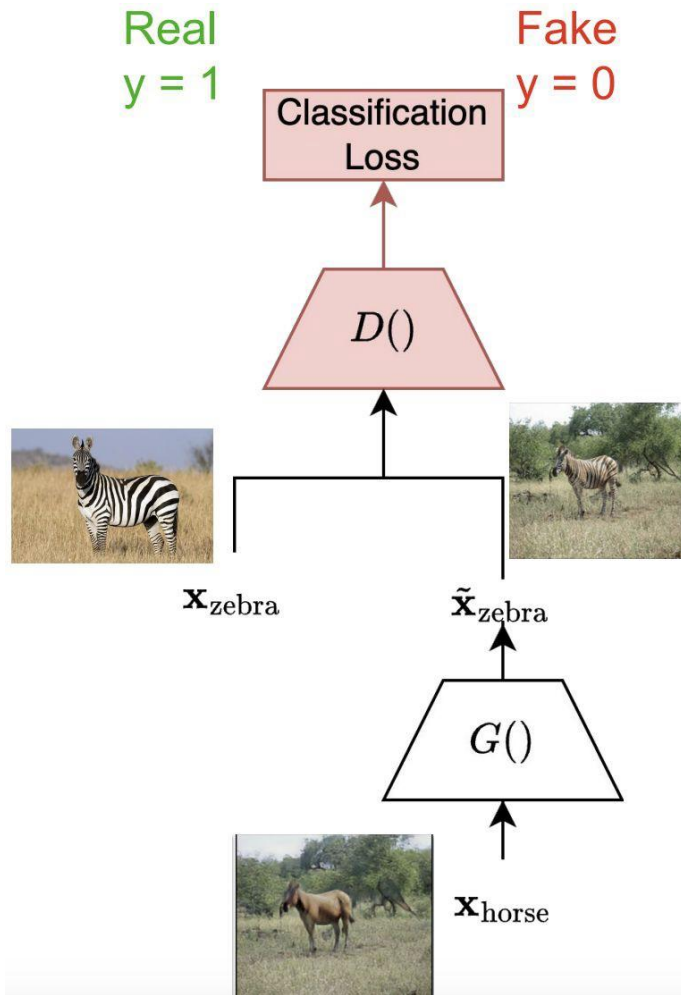
GANs

- Motivation: Conditional image generation without labelled pairs (Why? Data scarcity)
 - Eg. translate horses into zebras



[12 Astonishing Facts About Horses](#)

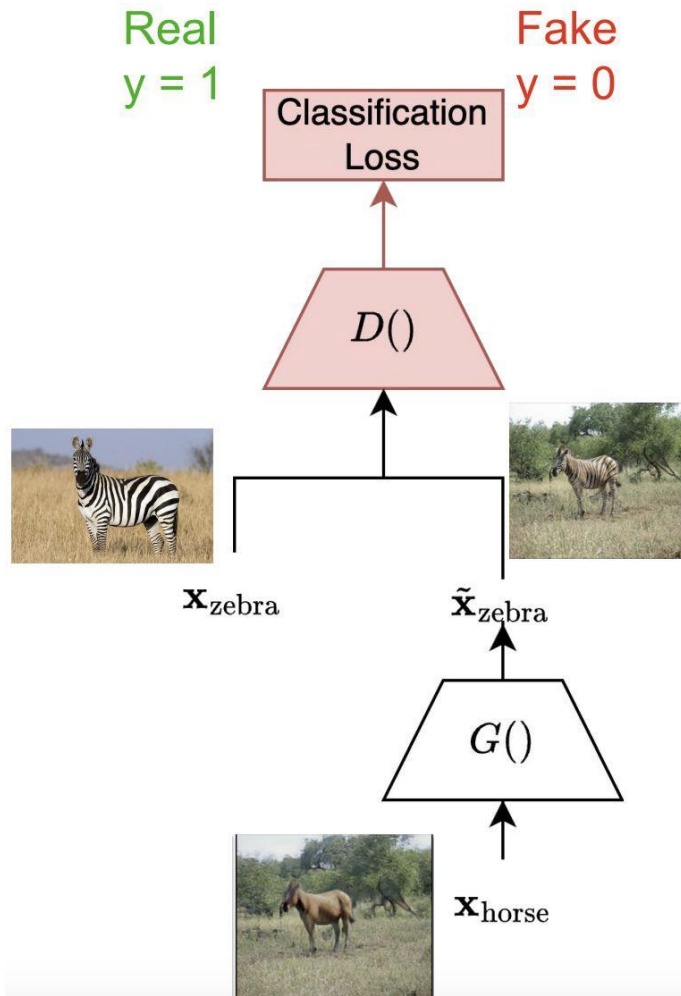
[Zebra Facts | Live Science](#)



GANs

Adversarial Networks

- Generator (G): performs the image translation
 - tries to *fool* the discriminator
- Discriminator (D): predicts whether the image was generated by the generator
 - measures how good the generator did



GANs

Positive example: real
image from labelled dataset

Negative example: fake
image from generator



Discriminator Loss:

$$L_D = \min_D [-\log(D(\mathbf{x})) - \log(1 - D(G(\mathbf{z})))]$$

Full GAN Loss:

$$\max_G \min_D [-\log(D(\mathbf{x})) - \log(1 - D(G(\mathbf{z})))]$$

GANs

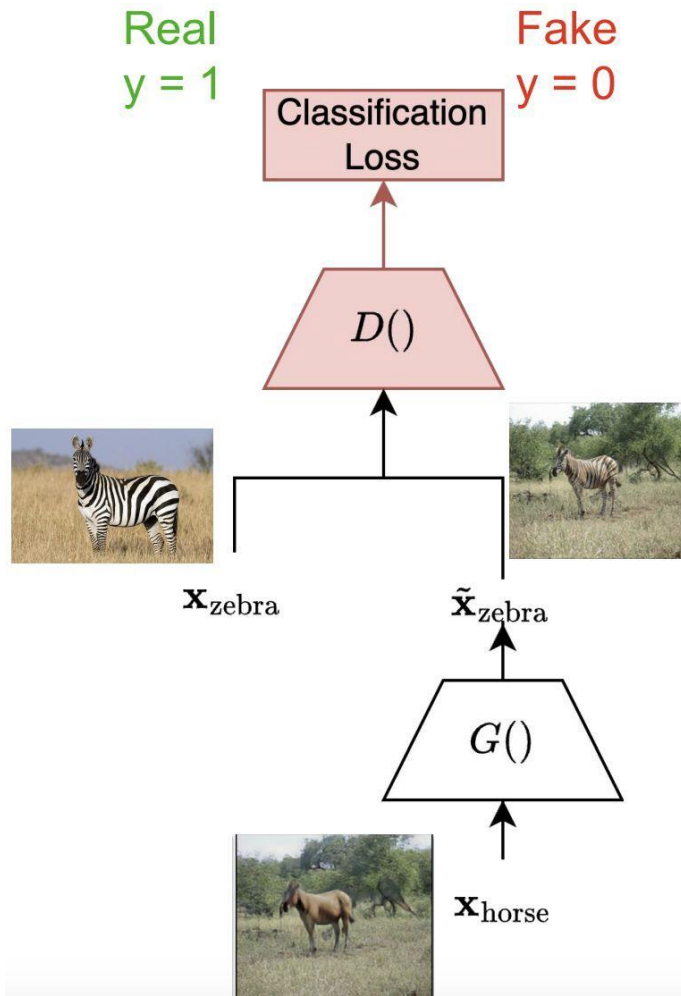
Adversarial Networks

- Generator (G): performs the image translation
 - tries to *fool* the discriminator
- Discriminator (D): predicts whether the image was generated by the generator
 - measures how good the generator did

To train, iteratively:

1. Update discriminator on real/fake images
2. Update generator using feedback from *new* discriminator

Why can't we train them simultaneously?



GANs

Issue 1: Minimax runs into vanishing gradient problems with a strong discriminator!

Minimax

$$\max_G [-\log(1 - D(G(\mathbf{z})))]$$

$$\min_G [\log(1 - D(G(\mathbf{z})))]$$

Minimizing generator
predicting **fake**

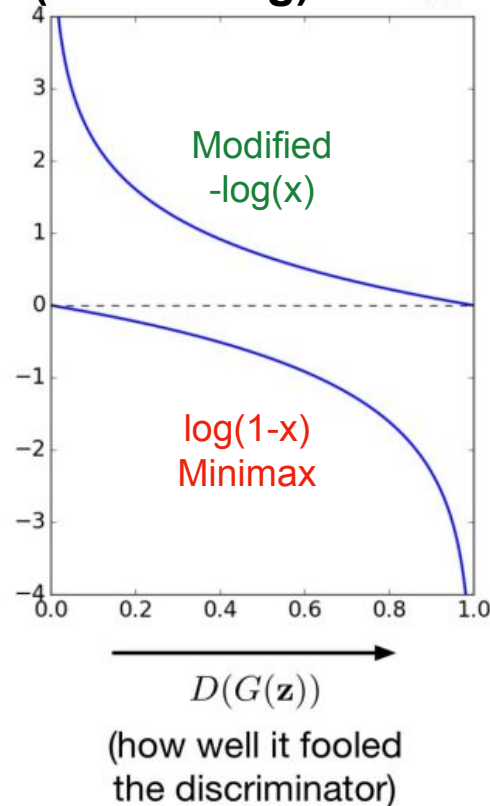
Modified

$$\max_G [\log(D(G(\mathbf{z})))]$$

$$\min_G [-\log(D(G(\mathbf{z})))]$$

Maximizing generator
predicting **real**

Loss (Minimizing)



GANs

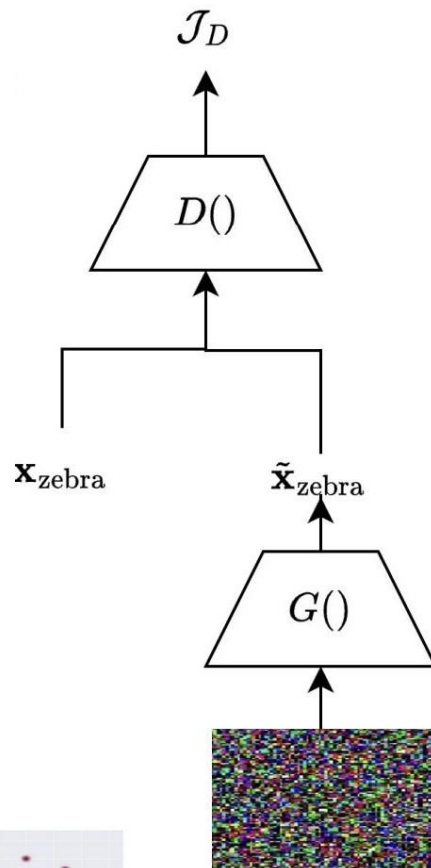
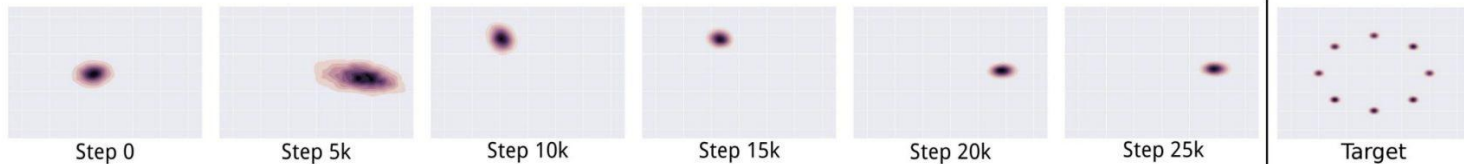
GANs can also be used for *unconditional* image generation

- Sample generator using Gaussian noise
- Results in very high quality images, quickly!

Issue 2: GANs experience Mode Collapse

- Learn only what best fools the discriminator
- Generated images do not represent full distribution of image class

Mode collapse

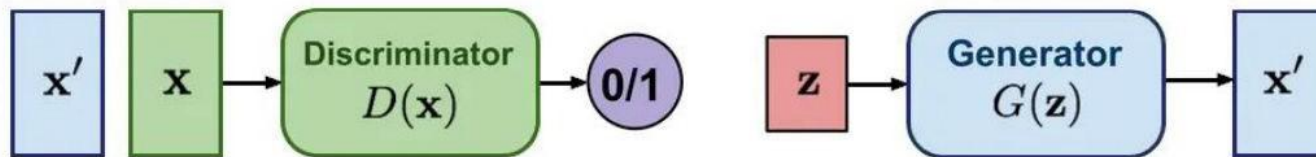


11. VAEs

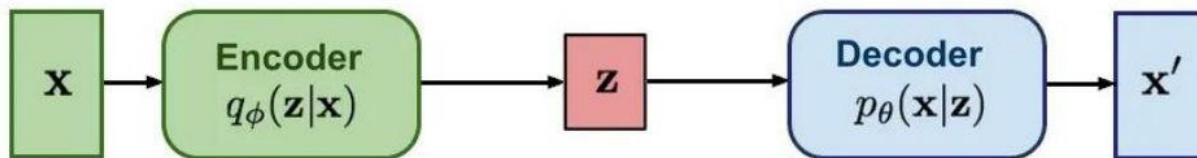


Variational Autoencoders (VAEs)

GAN: Adversarial training



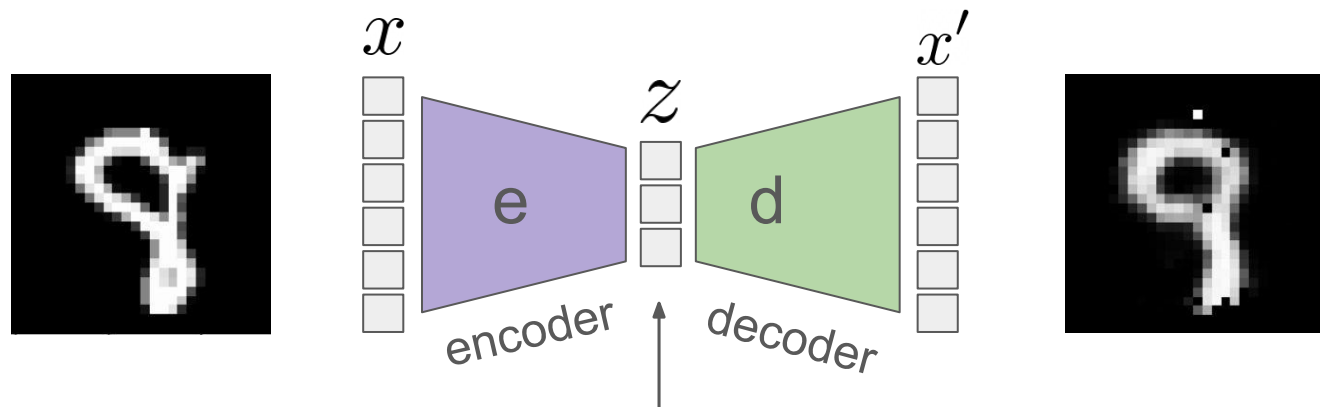
VAE: maximize variational lower bound



Diffusion models:
Gradually add Gaussian noise and then reverse



(Plain) Autoencoders

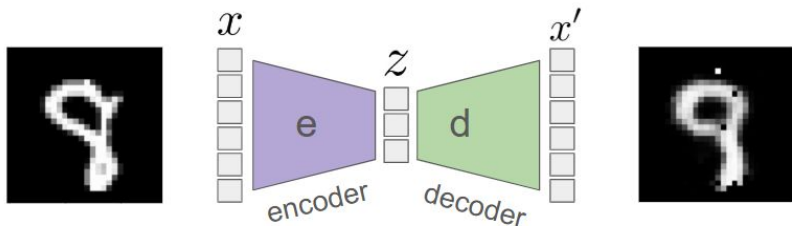


Latent space: entire space that z is in

Latent variable: a specific $z=e(x)$ produced by the encoder. We use this z to generate an image with the decoder.

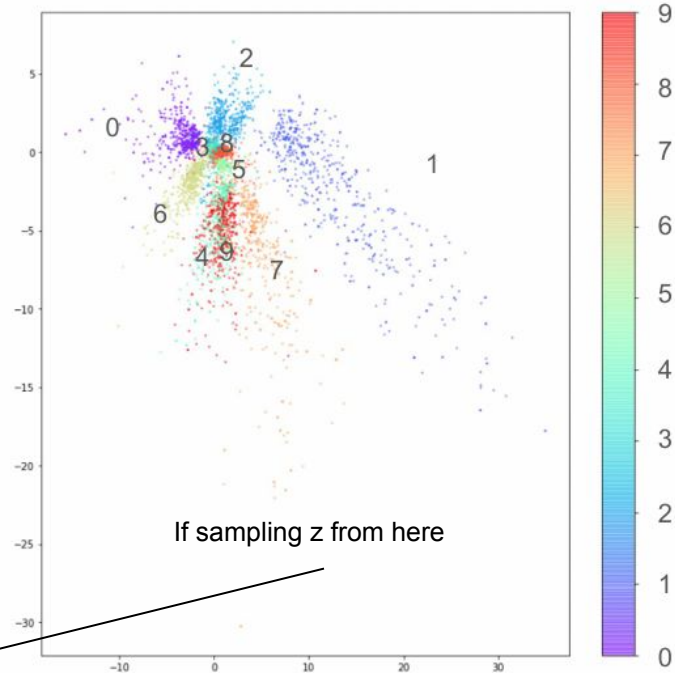
- Usually use Mean Squared Error $L = ||x - x'||^2$ as the reconstruction loss
- Train encoder and decoder with reconstruction loss, but then at test time only use the decoder

Problems with Plain Autoencoders



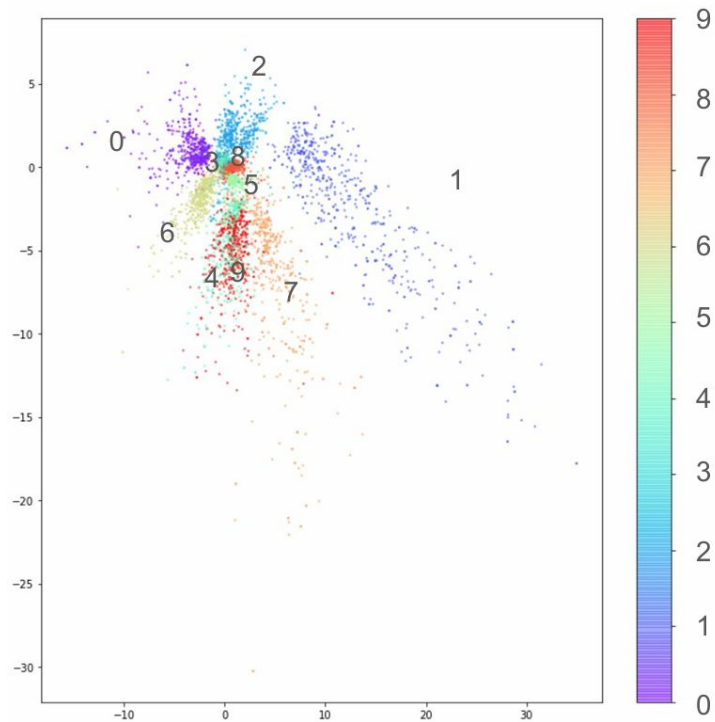
- No constraint for what latent space should look like (**VAEs change this aspect**)
- This means randomly sampling z in the latent space after training will most likely produce a garbage image (decoder has never been trained on this z before)

Latent Space of Autoencoder trained on MNIST



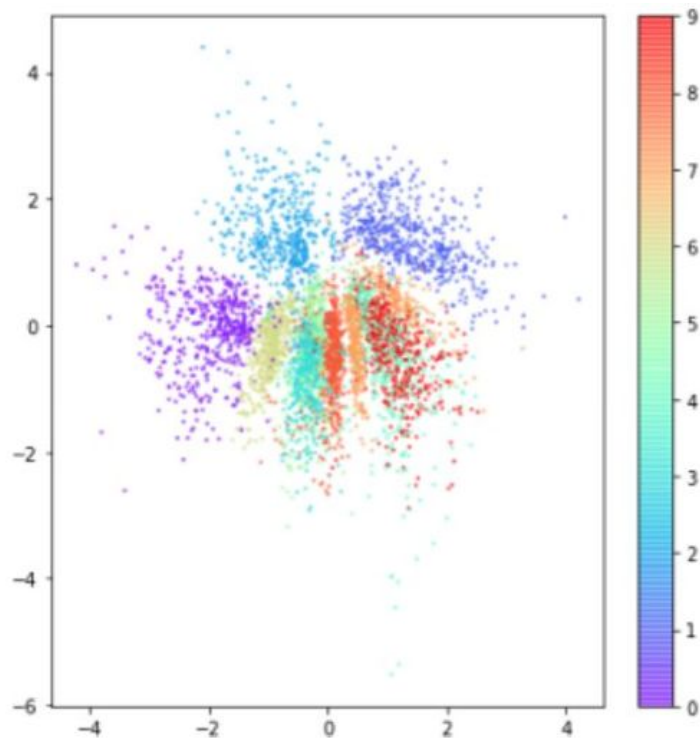
Get Garbage image

Latent space we have



Randomly distributed latent points

Latent space we want

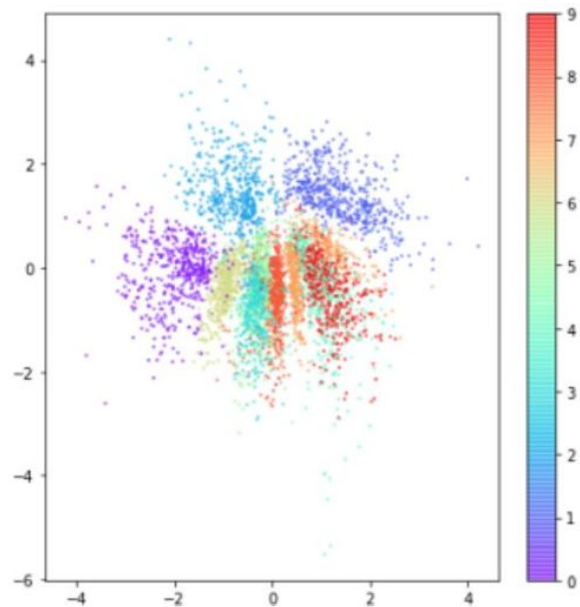
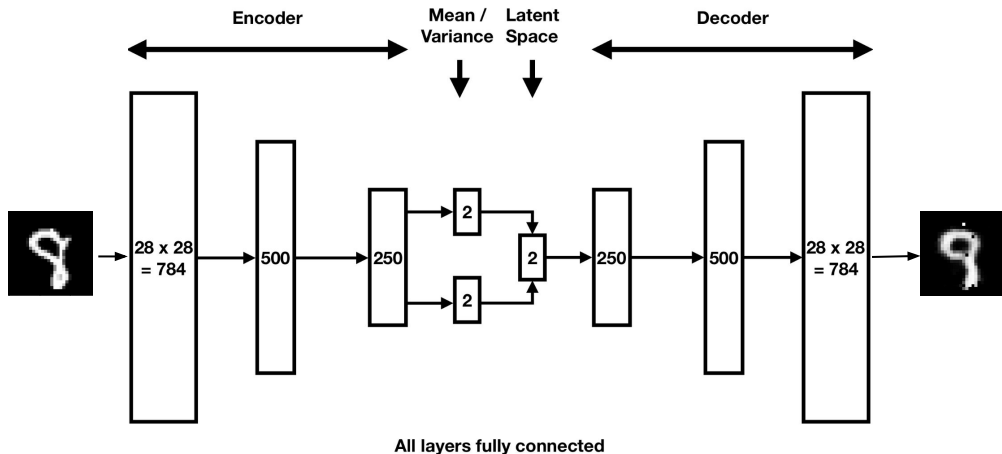


Standard normal distribution latent points

Solution: Variational Autoencoders (VAEs)

Want make the **latent space approximately the standard normal distribution** (**mean = 0, variance = 1**). This makes sampling more reliable because the latent points are in a compact region instead of some weird, funky distribution.

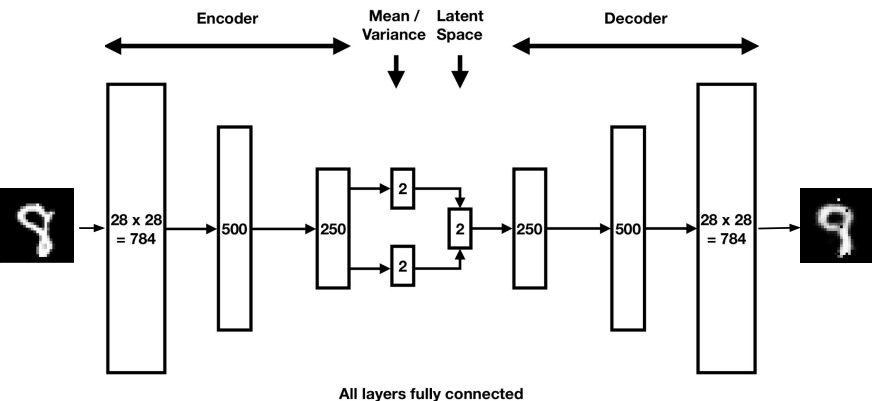
We want the VAE to look like this



Solution: Variational Autoencoders (VAEs)

Want make the **latent space** approximately the **standard normal distribution** (mean = 0, variance = 1).

We want the VAE to look like this



This means our training objective should try to **minimize**:

(Reconstruction loss) + KL (latent distribution || $N(0,1)$)

From plain autoencoder

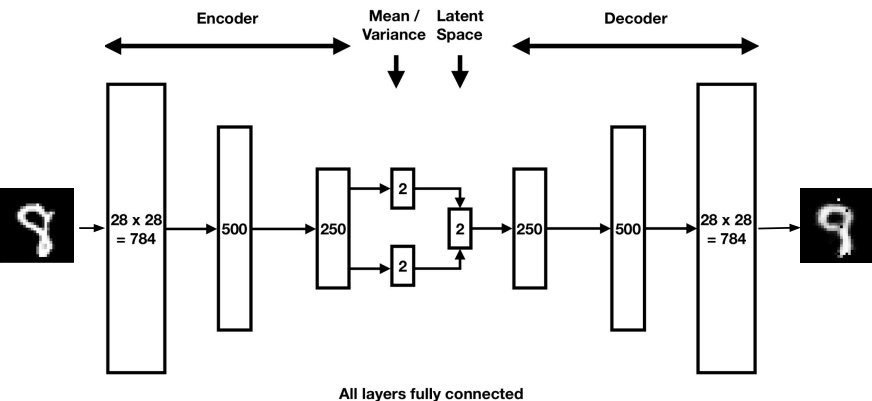
New to VAE

*Remember: KL divergence == 0 when two probability distributions are identical

Solution: Variational Autoencoders (VAEs)

Want make the **latent space** approximately the **standard normal distribution** (mean = 0, variance = 1).

We want the VAE to look like this



This means our training objective should try to **minimize**:

(Reconstruction loss) + KL (latent distribution || N(0,1))

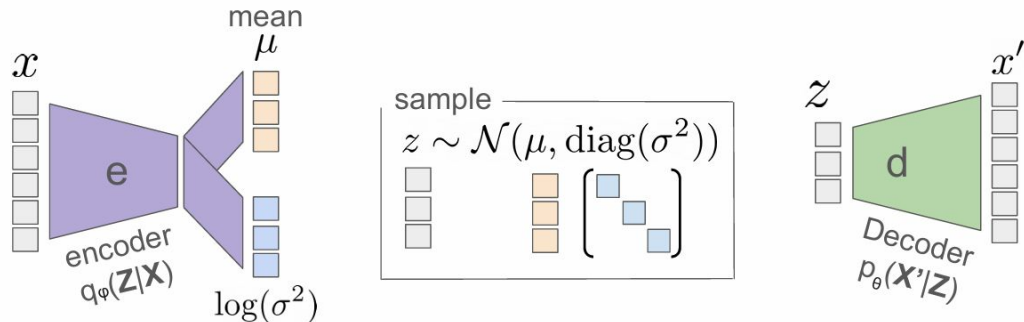
From plain autoencoder

New to VAE

$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[(\mathbf{x} - d_{\theta}(z))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(z|x) || p(z))}_{\text{prior matching term}}$$

*Remember: KL divergence == 0 when two probability distributions are identical

VAE: Encode latent space to match Stand. Norm. Distribution



Get vectors

$$\mu = (\mu_1, \mu_2, \dots, \mu_d), \quad \sigma^2 = (\sigma_1^2, \dots, \sigma_d^2)$$

Sample

$$z = (z_1, z_2, \dots, z_d)$$

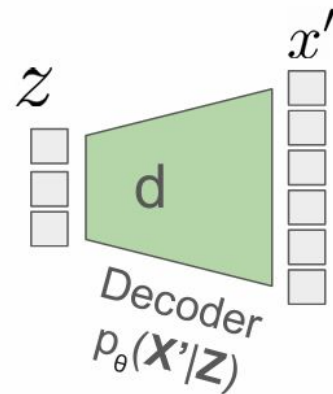
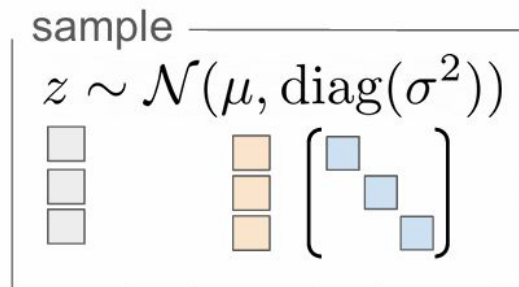
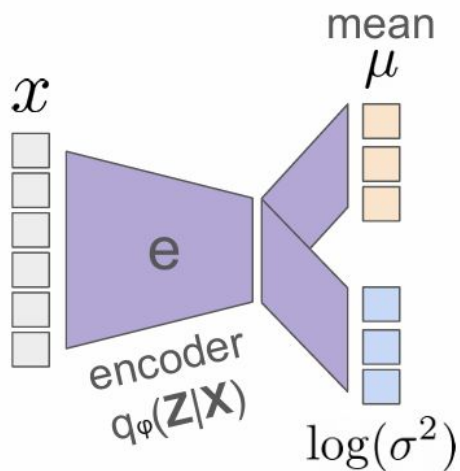
where each coordinate is sampled separately

$$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$$

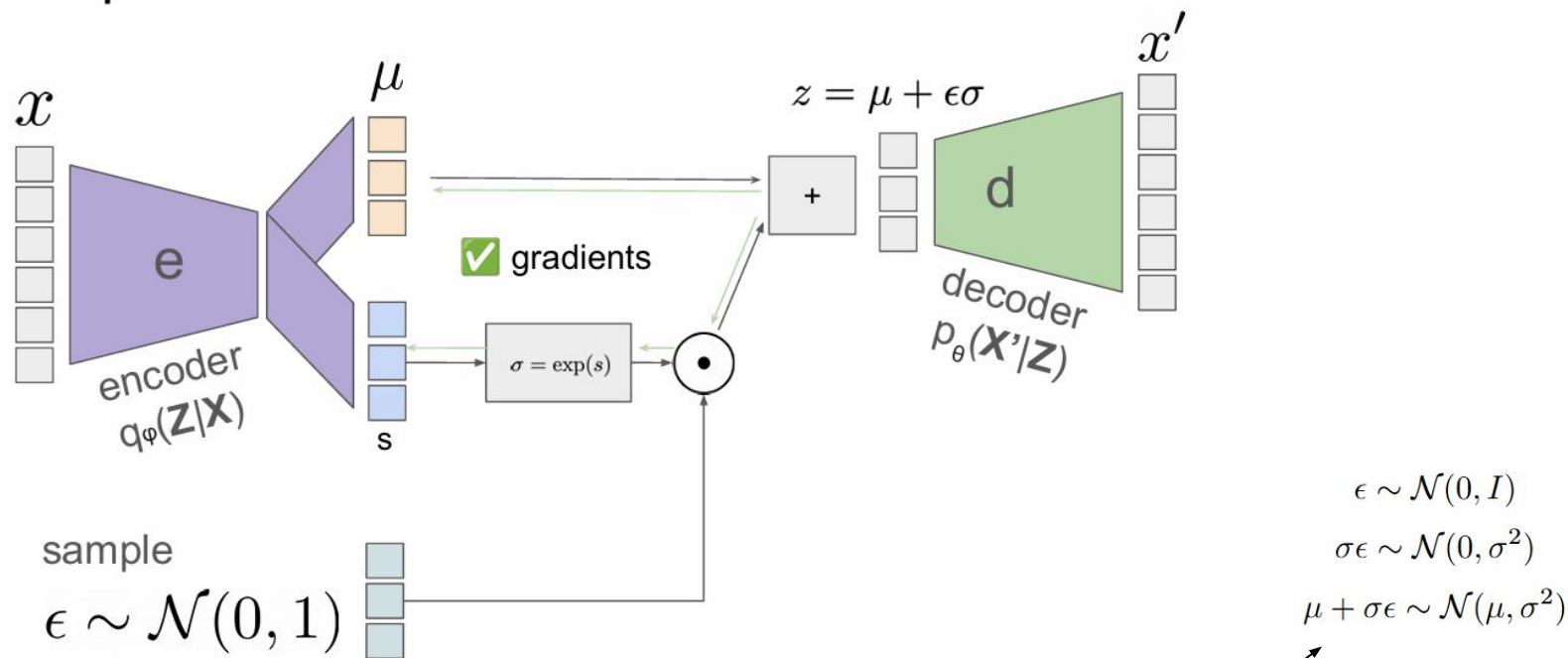
Problem: separate sampling causes issues during backpropagation, $z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ is not differentiable (because z_i is randomly chosen / not deterministic). Can't do gradient descent.

VAE: Reparameterization Trick

Solution to Backprop issue: Use the reparameterization trick to sample $\mathcal{N}(\mu, \text{diag}(\sigma^2)) = \mu + \sigma \odot \mathcal{N}(0, I)$

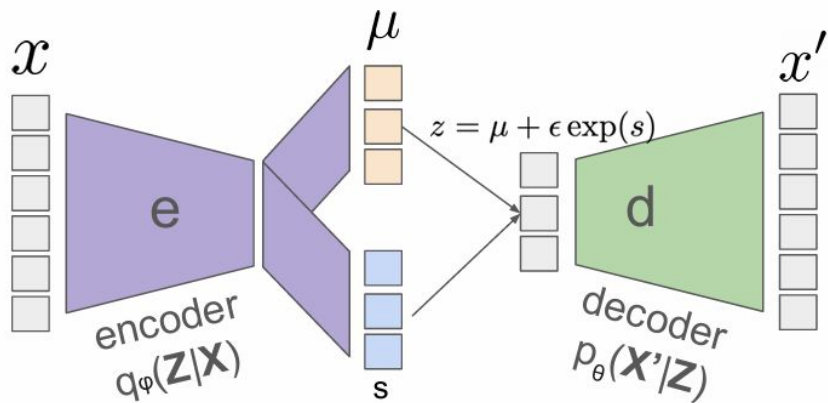


VAE: Reparameterization Trick



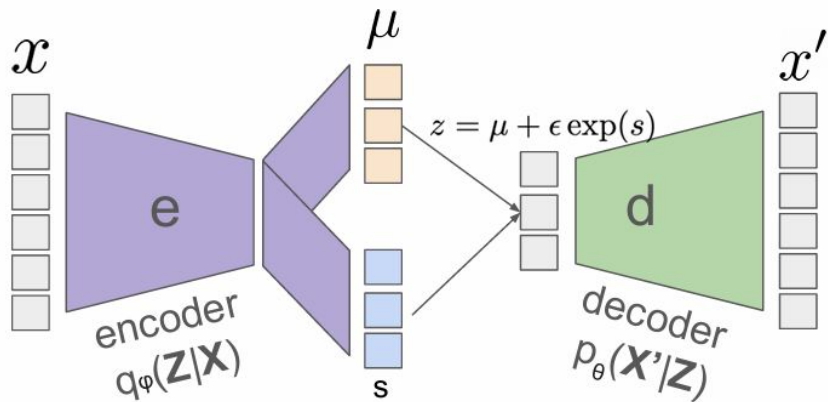
Latent variable z is still sampled from Gaussian Distribution AND we now have derivatives $\frac{\partial z}{\partial \mu}$ and $\frac{\partial z}{\partial \sigma}$ to use for backprop

VAE



Training objective:
$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[(\mathbf{x} - d_{\theta}(\mathbf{z}))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{prior matching term}}$$

VAE

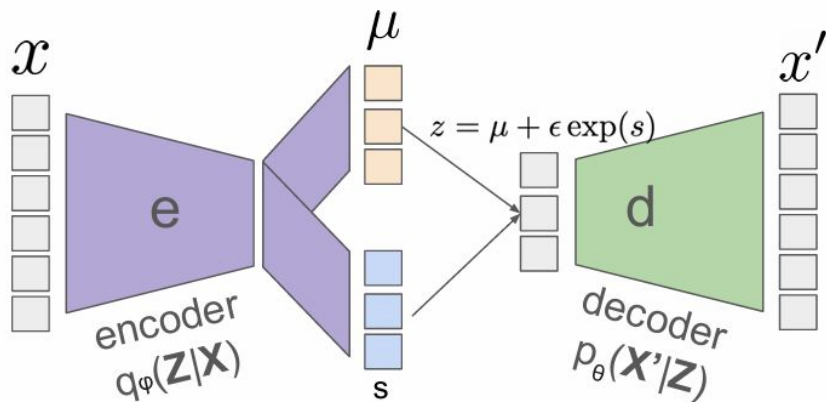


Training objective:

$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[(\mathbf{x} - d_{\theta}(z))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{prior matching term}}$$

Why not $D_{\text{KL}}(p(z) \| q_{\phi}(z|x))$?

VAE



Why not $D_{KL}(p(z)||q_\phi(z|x))$?

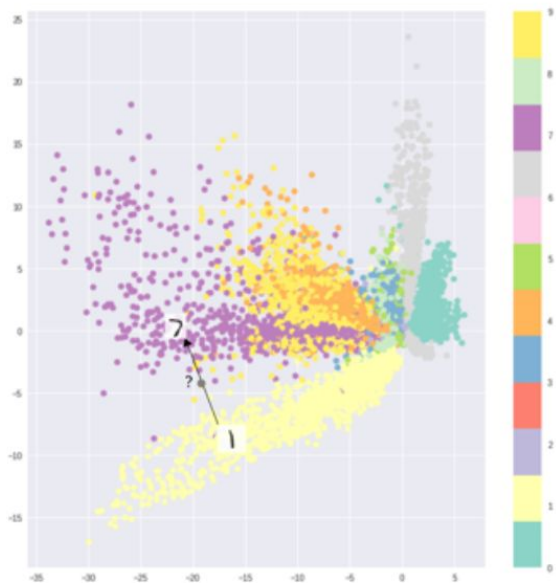
Answer: we would then be sampling from $\mathcal{N}(0, I)$ and trying to match the encoder to $q_\phi(z|x)$
Makes no sense.

Training objective:

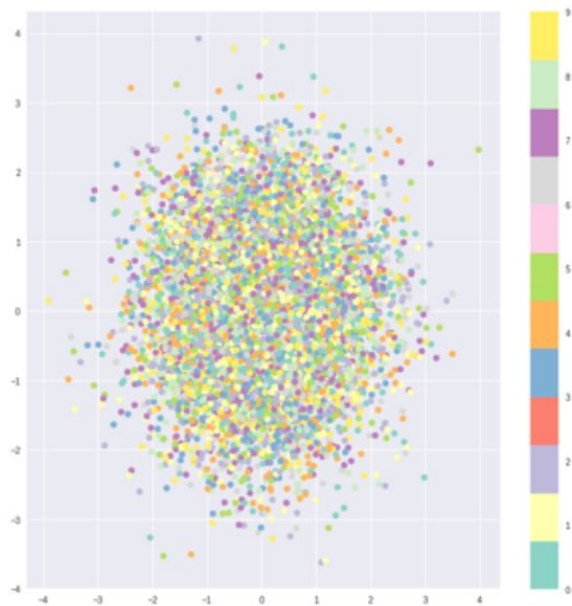
$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} [(\mathbf{x} - d_\theta(\mathbf{z}))^2]}_{\text{reconstruction term}} + \underbrace{D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{prior matching term}}$$

Why adding KL divergence helps

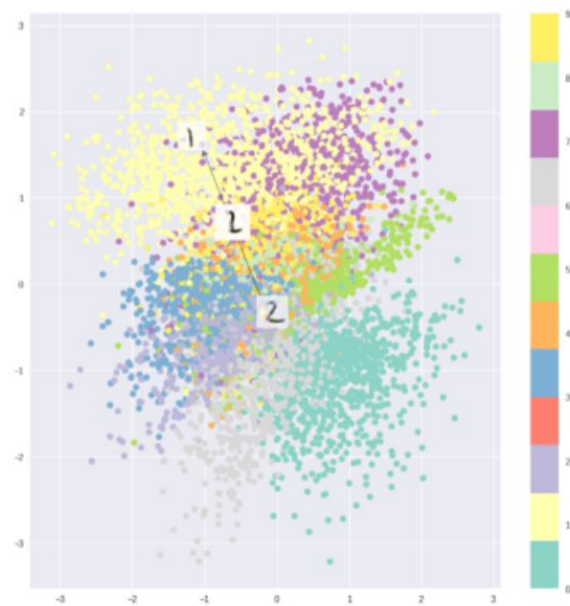
Only reconstruction loss



Only KL divergence



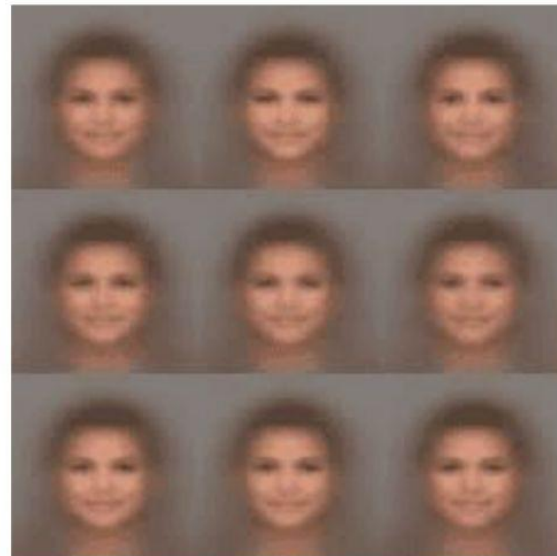
Combination



Drawbacks of VAEs

- Out of the box, generated images can be blurry.

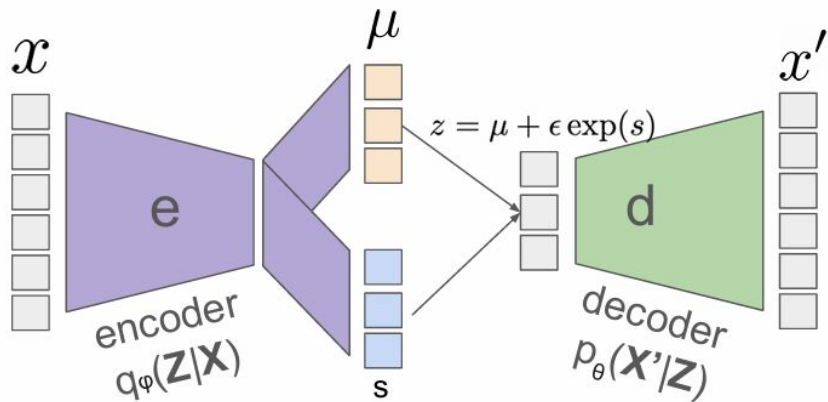
Question: Why? How do GANs fix this problem?



Training objective:

$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|\mathbf{x})} \left[(\mathbf{x} - d_{\theta}(z))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(z|\mathbf{x}) \| p(z))}_{\text{prior matching term}}$$

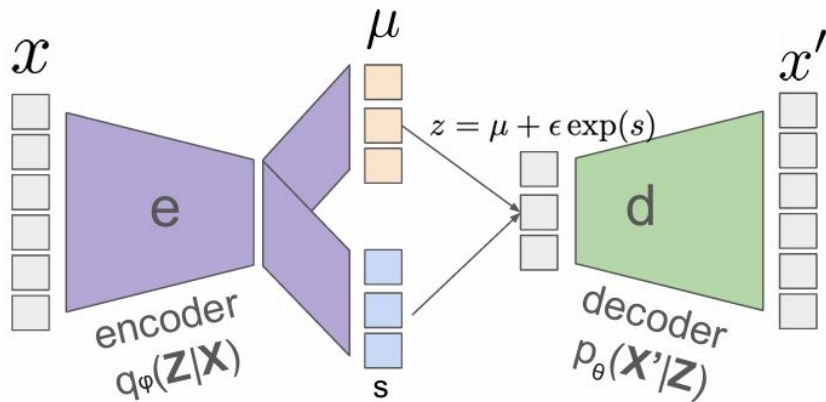
VAE: Intuition for ELBO



Training objective:

$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[(\mathbf{x} - d_{\theta}(\mathbf{z}))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{prior matching term}}$$

VAE: Intuition for ELBO



Training objective:

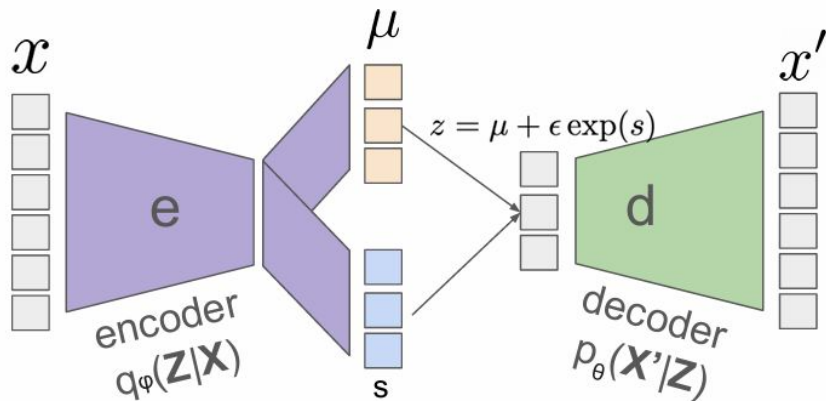
$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[(\mathbf{x} - d_\theta(\mathbf{z}))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{prior matching term}}$$

$$= \max_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{prior matching term}}$$

ELBO:

$$\log(P(x)) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{prior matching term}}$$

VAE: Intuition for ELBO



Remember: Training always seeks to maximize $\log(P(x))$

ELBO is saying: VAE training objective is $\leq$ (worse than/ at best equal to) the true optimal solution $\log(P(x))$

Training objective:

$$\min_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[(x - d_{\theta}(z))^2 \right]}_{\text{reconstruction term}} + \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{prior matching term}}$$

$$= \max_{\phi, \theta} \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{prior matching term}}$$

ELBO:

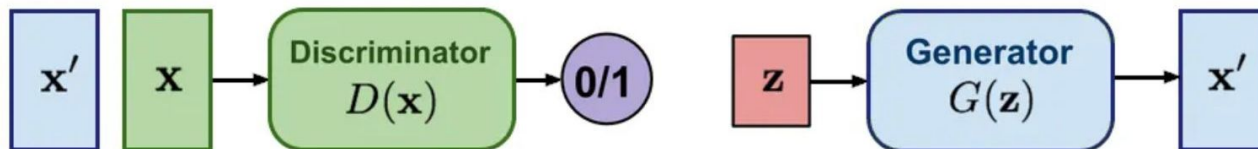
$$\log(P(x)) \geq \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_{\phi}(z|x) \| p(z))}_{\text{prior matching term}}$$

12. Diffusion Models

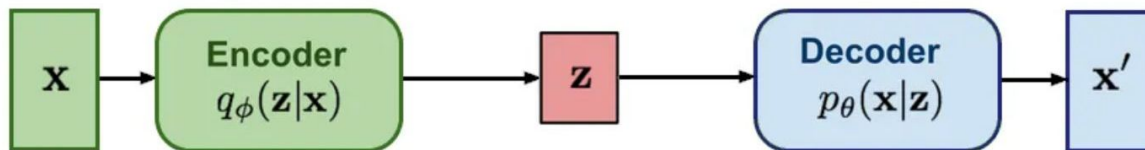


Diffusion Models

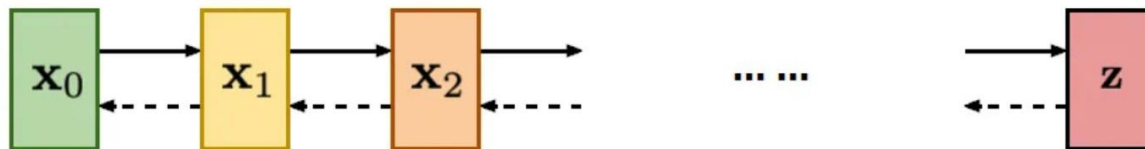
GAN: Adversarial training



VAE: maximize variational lower bound

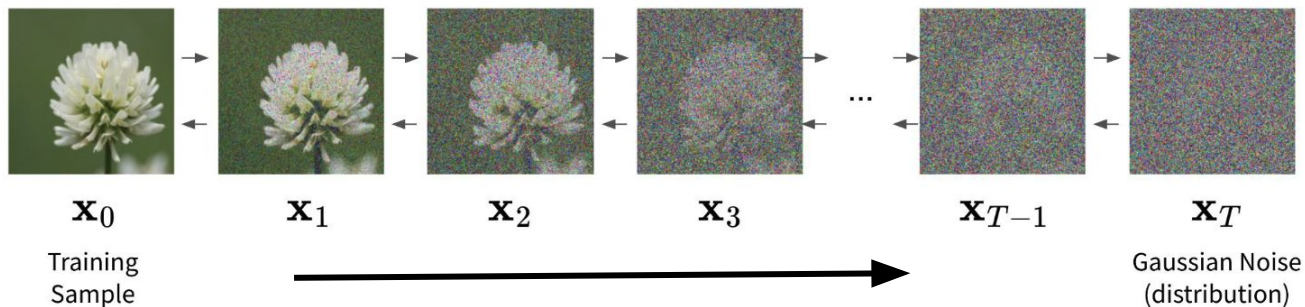


Diffusion models:
Gradually add Gaussian noise and then reverse

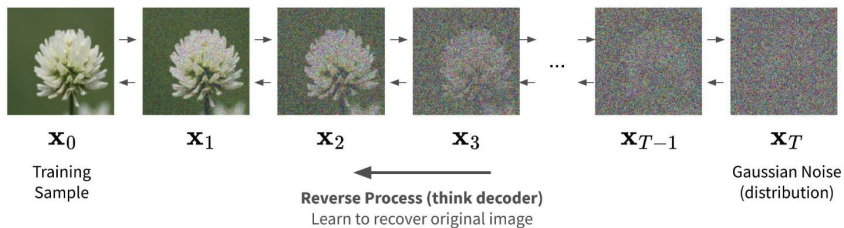


Diffusion Models - 2 parts

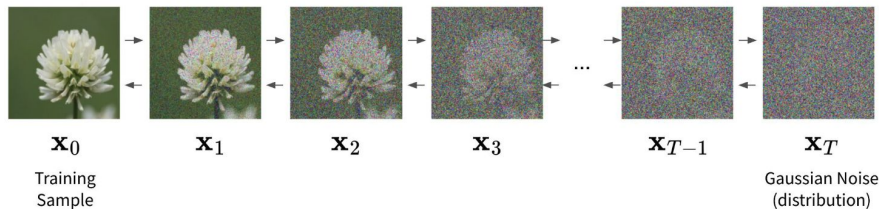
1) Forward process (progressively add noise)



2) Backward process (progressively denoise). This is what is actually used for inference in a Diffusion model

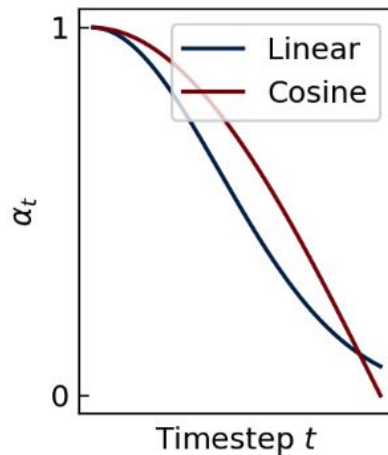


Forward Process

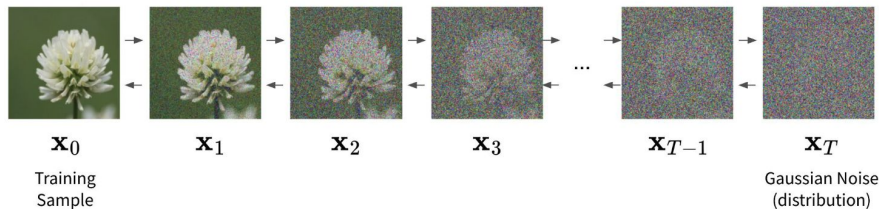


At each timestep, you add Gaussian noise according to a noise scheduler

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1} \quad \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$



Forward Process



At each timestep, you add Gaussian noise according to a noise scheduler

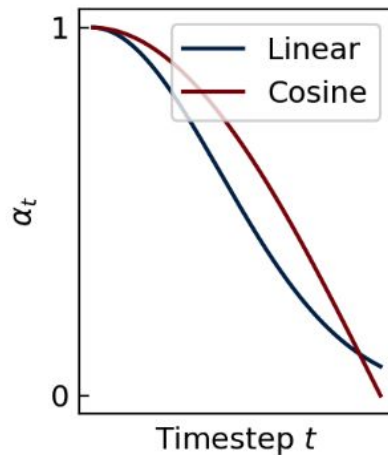
$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \quad \boldsymbol{\epsilon}_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\boldsymbol{\epsilon}}_{t-2} \quad \text{Merged noise. epsilon is still } \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

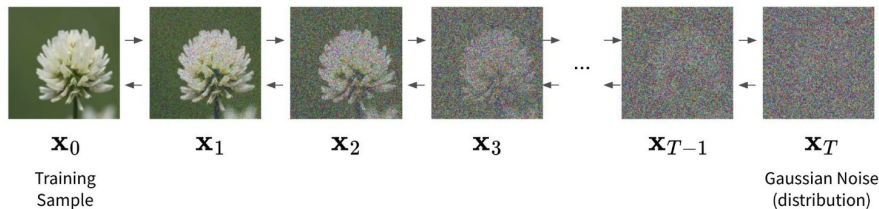
$= \dots$

$$= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$



Forward Process



At each timestep, you add Gaussian noise according to a noise scheduler

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}$$

$$\epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

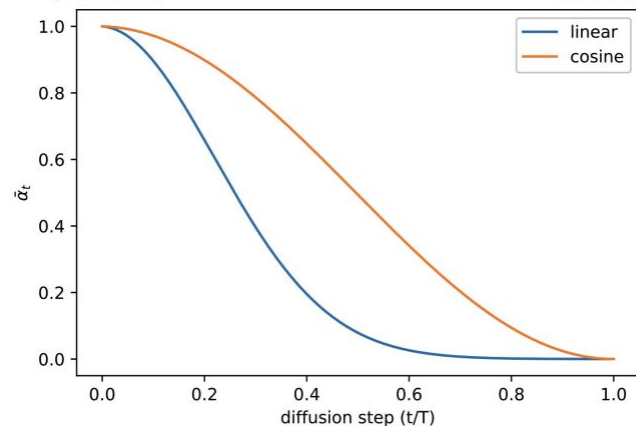
$$= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\epsilon}_{t-2}$$

Merged noise.
epsilon is still $\sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

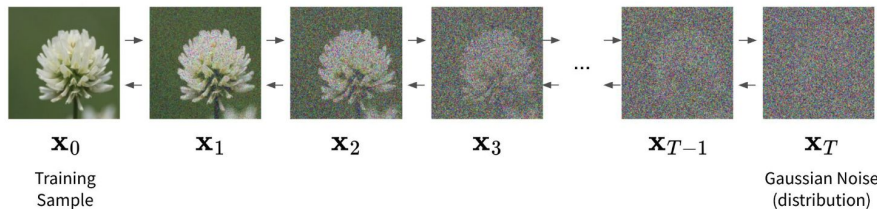
$$= \dots$$

$$= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

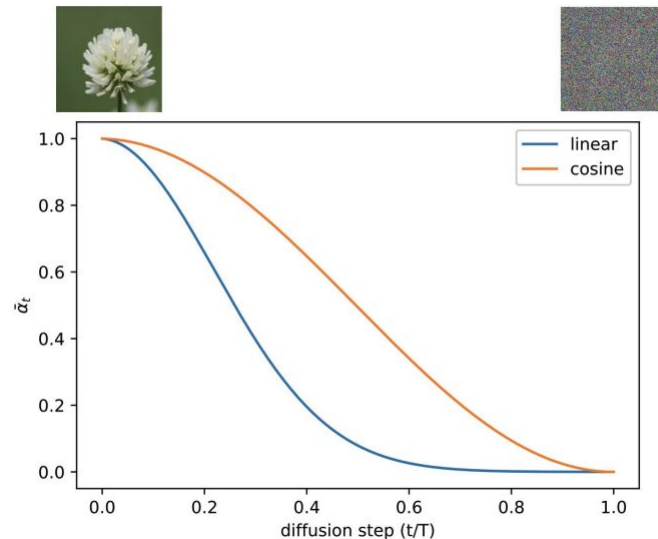


Forward Process



At each timestep, you add Gaussian noise

$$\begin{aligned}
 \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} & \boldsymbol{\epsilon}_{t-1} &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\
 &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\boldsymbol{\epsilon}}_{t-2} & \text{Merged noise.} \\
 &= \dots & \text{epsilon is still } \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\
 &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon} & \bar{\alpha}_t = \prod_{i=1}^t \alpha_i
 \end{aligned}$$



The final timestep goes to approximately standard normal distribution

$$\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Training Algo for Diffusion Model (noise prediction)

For each training iteration:

1. Pick random training image x_0
2. Pick a random timestep t
3. Get noisy image x_t from $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, I)$
4. Pass noisy image x_t and timestep t into UNet to get its prediction

$$\varepsilon_{\theta}(x_t, t)$$

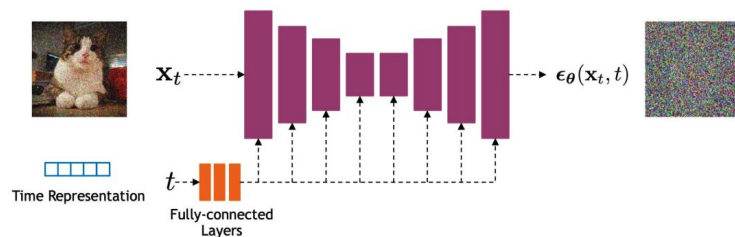
of the image ε

5. Compute loss

$$L(\theta) = \|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2$$

and backprop through the UNet

Network for the diffusion model is usually a UNet (same input and output dimensions)



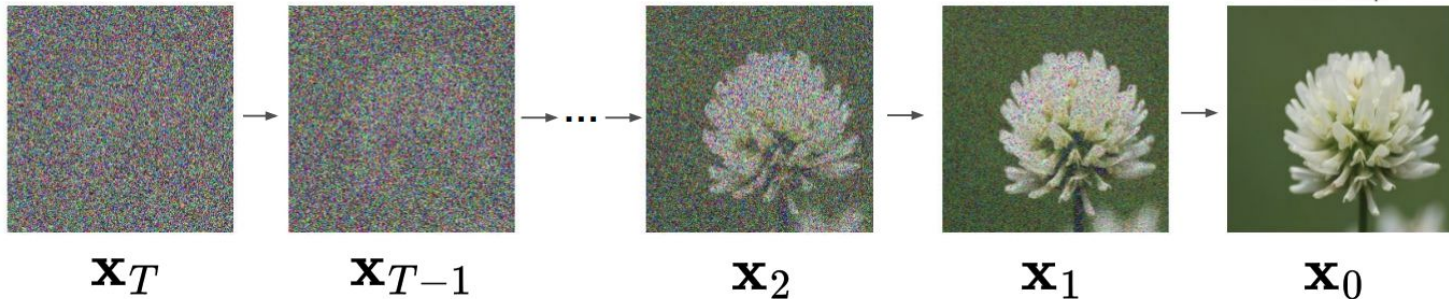
- People often use U-Nets with residual blocks, self-attention, cross attention
- Time representation: sinusoidal positional embeddings. Add time embeddings in the residual blocks throughout the UNet
- Text Conditioning (not shown in image above): text goes through text encoder and used in cross attention

Backward Process

Start from $q(\mathbf{x}_T) = \mathcal{N}(0, \mathbf{I})$

Can we sample from reversed conditional distribution $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$

How to compute $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$?

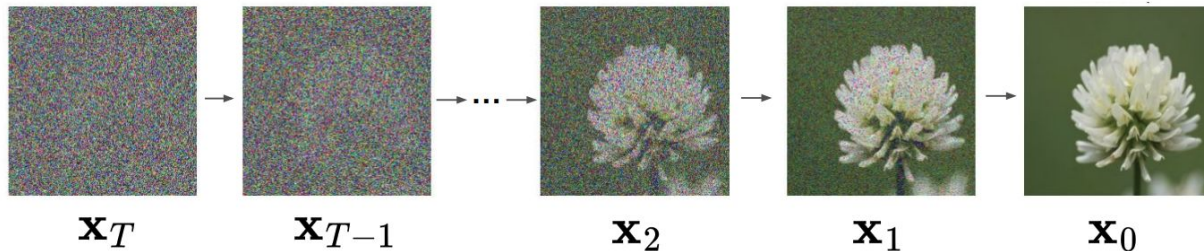


Backward Process

- Could try Bayes Rule
- But the formula is **Intractable** (too hard to compute in practice) because need to integrate over the entire data distribution (all possible noised versions of the image)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$


$$q(\mathbf{x}_t) = \int q(\mathbf{x}_0)q(\mathbf{x}_1|\mathbf{x}_0)\dots q(\mathbf{x}_t|\mathbf{x}_{t-1})d\mathbf{x}_0d\mathbf{x}_1\dots d\mathbf{x}_{t-1}$$



Backward Process

- There's a trick, we have $\mathbf{x}_0$ during training

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$



$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

This denoising equation is now feasible
computable (tractable)!

From forward process

$$\begin{aligned}\mathbf{x}_t &= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ q(\mathbf{x}_t|\mathbf{x}_0) &= \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})\end{aligned}$$

Sampling Algo (running Diffusion Model at inference time)

DDIM Sampling (covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Estimate the clean image

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

3. Update to the next less-noisy sample

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_\theta(x_t, t)$$

Return x_0

$$x_t = \sqrt{\bar{\alpha}_t} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)$$

Just algebraic manipulation from forward process

Sampling Algo (running Diffusion Model at inference time)

DDIM Sampling (covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Estimate the clean image

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

3. Update to the next less-noisy sample

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_\theta(x_t, t)$$

Return x_0

DDPM Sampling (not covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Sample Gaussian noise for this reverse step

$$z \sim \mathcal{N}(0, I)$$

3. Update to the next less-noisy sample

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z \quad (\sigma_t \text{ comes from noise scheduler})$$

Return x_0

Sampling Algo (running Diffusion Model at inference time)

DDIM Sampling (covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Estimate the clean image

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

3. Update to the next less-noisy sample

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_\theta(x_t, t)$$

Return x_0

- Deterministic (less diverse) output once x_T is chosen
- Faster to generate reasonable image (can use lower T)

DDPM Sampling (not covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Sample Gaussian noise for this reverse step

$$z \sim \mathcal{N}(0, I)$$

3. Update to the next less-noisy sample

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z \quad (\sigma_t \text{ comes from noise scheduler})$$

Return x_0

- Non-deterministic (more diverse) image output once x_T is chosen
- Slower to generate reasonable image (need higher T)

Sampling Algo (running Diffusion Model at inference time)

DDPM Sampling (not covered in lecture)

Start from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

For $t = T, T-1, \dots, 1$:

1. Pass x_t and t to UNet to predict the added noise

$$\varepsilon_\theta(x_t, t)$$

2. Sample Gaussian noise for this reverse step

$$z \sim \mathcal{N}(0, I)$$

3. Update to the next less-noisy sample

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z \quad (\sigma_t \text{ comes from noise scheduler})$$

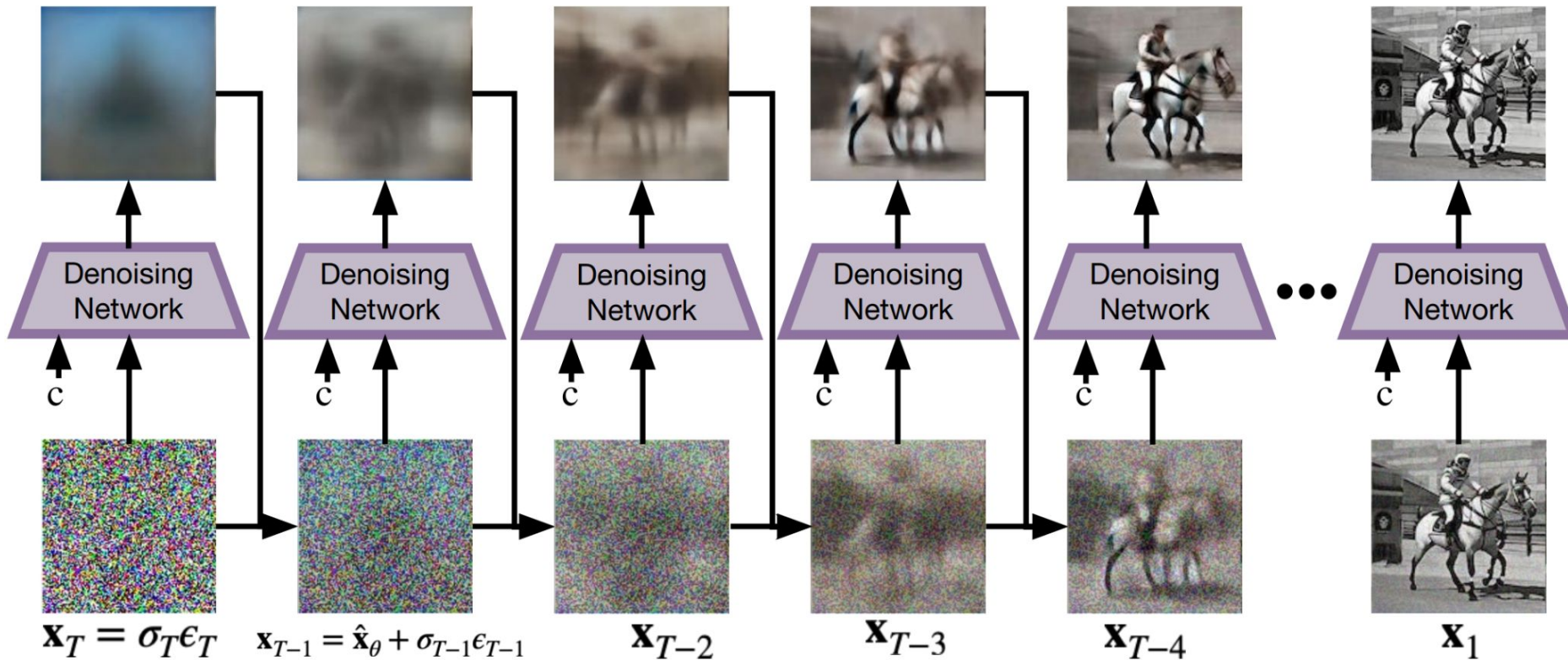
Return x_0

Long probability derivation for how to get this update equation, but it comes from our tractable equation two slides earlier

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)}$$

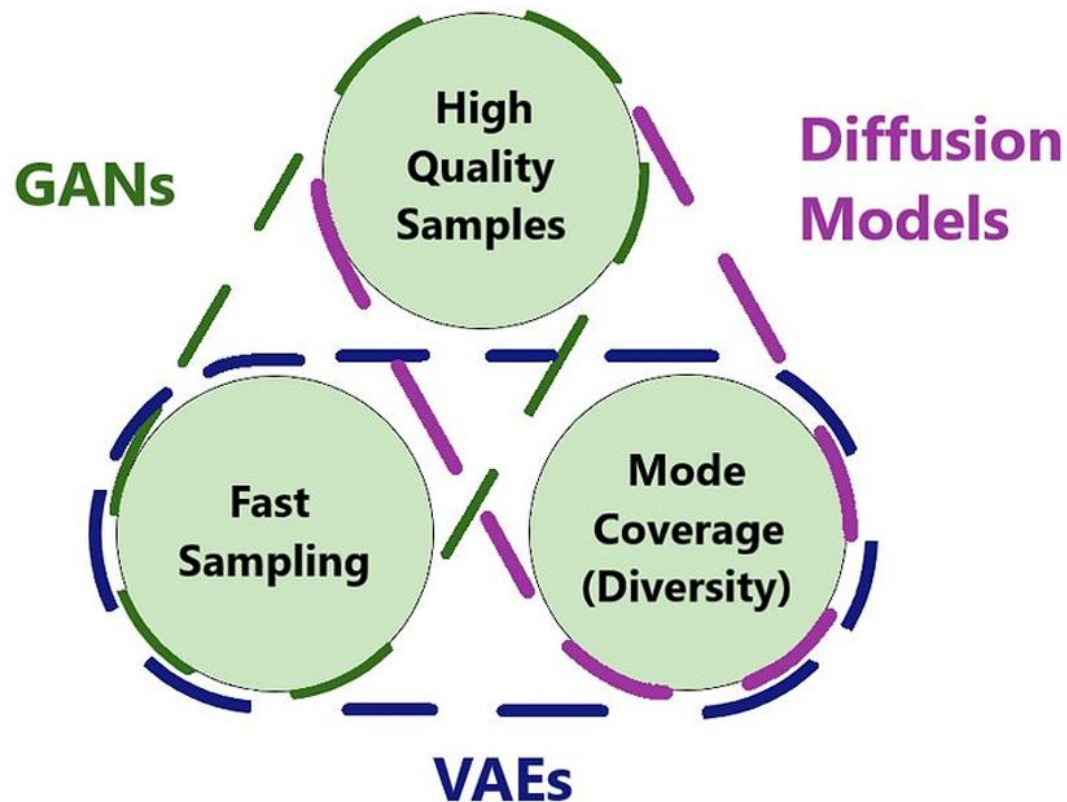
Diffusion (DDPM Sampling) Visualization

C = "An astronaut riding on a horse"



DDIM Sampling would look very similar to this diagram

Summary:



Additional Practice Problems

Test Your Transformer Knowledge!

Spring 2025 Midterm

1. (2) Explain why positional embeddings are needed for Transformers but not for RNNs.
2. (4) Instead of specifying the positional embedding (as a combination of sine and cosine functions), another option is to *learn* the positional vectors. This is common practice with Vision Transformers, but not so common for text input. Explain why learned vectors might be problematic in the context of text.
3. (T/F) One of the advantages of training Transformers compared to LSTM models is that Transformers are parallelizable during training.

Problem 2

For each of the following statements, indicate whether the statement describes VAEs, GANs, or Diffusion models. You should list all that apply.

1. Trained with a likelihood-based objective.
2. Has a learnable encoder that maps the data to a Gaussian distribution.
3. Has latent variables that must be the same size as the input data.
4. Learns to generate images by fooling a discriminator.
5. Generates high-quality, realistic images.
6. Generates images with a single forward pass of the network.
7. Often suffers from poor diversity among generations.
8. Transforms samples from a unit normal distribution to samples from the data distribution.

Problem 2

For each of the following statements, indicate whether the statement describes VAEs, GANs, or Diffusion models. You should list all that apply.

- | | |
|---|--|
| Y | 1. Trained with a likelihood-based objective. |
| N | 2. Has a learnable encoder that maps the data to a Gaussian distribution. |
| Y | 3. Has latent variables that must be the same size as the input data. |
| N | 4. Learns to generate images by fooling a discriminator. |
| Y | 5. Generates high-quality, realistic images. |
| N | 6. Generates images with a single forward pass of the network. |
| N | 7. Often suffers from poor diversity among generations. |
| Y | 8. Transforms samples from a unit normal distribution to samples from the data distribution. |

9. CLIP is a constrastive learning algorithm that learns to map paired text and images close together in a shared embedding space.

10. Vision transformers typically require less training data than convolutional neural networks (CNNs) to achieve comparable performance on image classification tasks.

9. CLIP is a constrastive learning algorithm that learns to map paired text and images close together in a shared embedding space.

Solution: True.

10. Vision transformers typically require less training data than convolutional neural networks (CNNs) to achieve comparable performance on image classification tasks.

Solution: False. Vision transformers often require larger amounts of training data compared to CNNs to achieve comparable performance on image classification tasks. CNNs have inductive biases such as translation invariance and local receptive fields, which allow them to learn effective features with less training data. However, when trained on sufficiently large datasets, vision transformers have shown impressive performance and have even outperformed CNNs on various benchmarks.

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

a) Adhitya has already written code to divide the input images of shape $(h \times w \times 3)$ into k patches and feed the patches to a ViT to produce 128-dimensional features for every patch. The hidden dimension of the decoder is 128. What are the dimensions of the weight matrices W_Q, W_K, W_V in the cross-attention layer?

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

a) Adhitya has already written code to divide the input images of shape $(h \times w \times 3)$ into k patches and feed the patches to a ViT to produce 128-dimensional features for every patch. The hidden dimension of the decoder is 128. What are the dimensions of the weight matrices W_Q, W_K, W_V in the cross-attention layer?

The dimensions of the weight matrices W_Q, W_K, W_V in the cross-attention layer are:

$W_Q : (128, 128)$ $W_K : (128, 128)$ $W_V : (128, 128)$

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

b) When you are predicting the first word, what are the shapes of the output matrices obtained after multiplying W_Q , W_K , W_V with their appropriate inputs.

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

b) When you are predicting the first word, what are the shapes of the output matrices obtained after multiplying W_Q, W_K, W_V with their appropriate inputs.

The shapes of the output matrices after multiplying W_Q, W_K, W_V with their appropriate inputs are:

$Q : (1, 128)$ - The query matrix is obtained by multiplying the decoder's input embedding (or the previous layer's output) with W_Q . $K : (k, 128)$ - The key matrix is obtained by multiplying the ViT's output features with W_K . $V : (k, 128)$ - The value matrix is obtained by multiplying the ViT's output features with W_V .

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

c) Suppose the ViT model produced 64-dimensional features for every patch (instead of 128-dimensional features), what would you have to modify to attend to these image features?

Problem 9: Image Captioning [9 pts]

Adhitya wants to train a model for image captioning. He plans to train a decoder-only transformer model that generates a natural language description by attending to visual features extracted from an input image. He has access to a pretrained ViT model to extract image features. Adhitya has written most of the network architecture and needs a little help with the cross-attention layer in the decoder (the cross-attention layer is used to attend to the image features).

c) Suppose the ViT model produced 64-dimensional features for every patch (instead of 128-dimensional features), what would you have to modify to attend to these image features?

Add a linear layer to project the embeddings to 128 dimensions.

Prelim Q2 (Only focus on GANs)

For each of the following statements, indicate whether the statement describes VAEs, GANs, or Diffusion models. You should list all that apply.

1. Trained with a likelihood-based objective.
2. Has a learnable encoder that maps the data to a Gaussian distribution.
3. Has latent variables that must be the same size as the input data.
4. Learns to generate images by fooling a discriminator.
5. Generates high-quality, realistic images.
6. Generates images with a single forward pass of the network.
7. Often suffers from poor diversity among generations.
8. Transforms samples from a unit normal distribution to samples from the data distribution.

Prelim Q2 (Only focus on GANs)

For each of the following statements, indicate whether the statement describes VAEs, GANs, or Diffusion models. You should list all that apply.

1. Trained with a likelihood-based objective.

N

2. Has a learnable encoder that maps the data to a Gaussian distribution.

N

3. Has latent variables that must be the same size as the input data.

N

4. Learns to generate images by fooling a discriminator.

Y

5. Generates high-quality, realistic images.

Y

6. Generates images with a single forward pass of the network.

Y

7. Often suffers from poor diversity among generations.

Y

8. Transforms samples from a unit normal distribution to samples from the data distribution.

Y

[End]